

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKIH
TEHNOLOGIJA OSIJEK

**Sveučilišni diplomski studij Računarstvo; Informacijske i podatkovne
znanosti**

WEB APLIKACIJA ZA ODRŽAVANJE E-SJEDNICE

Diplomski rad

Luka Šaško

Osijek, 2025

SADRŽAJ

1. UVOD	1
2. PREGLED PODRUČJA TEME	2
3. OPIS SUSTAVA	3
3.1. Arhitektura web aplikacije.....	3
3.2. Korištene tehnologije	3
3.3. Opis poslužiteljske strane.....	4
3.3.1. Entiteti baze podataka	4
3.3.2. Autentifikacija i autorizacija	4
3.3.3. Servisni sloj	5
3.4. Opis korisničkog sučelja.....	5
3.4.1. Upravljanje stanjem pomoću MobX-a	5
3.4.2. REST API komunikacija	6
3.4.3. WebSocket integracija.....	6
3.4.4. Ključne komponente korisničkog sučelja	6
4. REZULTATI IMPLEMENTACIJE	7
4.1. Implementacija baze podataka	7
4.2. Implementacija Backend dijela aplikacije	10
4.2.1. Sloj modela	10
4.2.2. Sigurnosni sloj	12
4.2.3. Kontrolni sloj.....	15
4.2.4. Servisni sloj	19
4.3. Implementacija frontend dijela aplikacije	22
4.3.1. Servisna klasa za stvaranje API poziva	23
4.3.2. Upravljanje stanjima pomoću MobX store	24
4.3.3. Komponentne i konkretne stranice	25
4.3.4. Dizajn stranice.....	32
5. PRIKAZ RADA APLIKACIJE.....	34
6. ZAKLJUČAK.....	45
LITERATURA.....	46
SAŽETAK	47
ABSTRACT.....	48
PRILOZI.....	49

1. UVOD

U današnje vrijeme velik broj organizacija i institucija pronalazi potrebu za prelazak na elektronično vođenje sjednica zbog učinkovitijeg upravljanja vremenom, veće dostupnosti i preglednosti podataka te mogućnosti sudjelovanja na daljinu. Glavni problemi tradicionalnih sjednica su zahtijevanje fizičke prisutnosti svih sudionika, što je organizacijski vrlo zahtjevno kada sudionici dolaze s različitih lokacija. Elektroničke sjednice automatiziraju procese odlučivanja, vođenja zapisnika i glasanja, čime se značajno pojednostavljuje upravljanje sjednicama.

Tema ovog diplomskog rada je razvoj web aplikacije za održavanje e-sjednica koja omogućava upravljanje elektroničkim sjednicama, stvaranje dnevnog reda, provođenje glasanja po točkama i prikaz grafičkih rezultata glasanja. Aplikacija je razvijena korištenjem Java programskog jezika i Spring Boot okvira za izradu backend sustava, dok je frontend aplikacije izrađen korištenjem ReactJS biblioteke uz podršku za komunikaciju u stvarnom vremenu putem WebSocket protokola.

Kroz rad su implementirane suvremene tehnologije razvoja web aplikacija, REST API za komunikaciju između klijenta i servera, JWT („JSON Web Token“) autentifikacija za upravljanje sigurnošću te notifikacije unutar stvarnoga vremena putem WebSocket tehnologije za informiranje sudionika o aktivnim glasanjima.

Rad je podijeljen u nekoliko poglavlja. Nakon uvodnog poglavlja, u drugom poglavlju daje se pregled područja i primjena sličnih rješenja. Treće poglavlje detaljno opisuje implementaciju sustava, uključujući korištene tehnologije i arhitekturu sustava. U četvrtom poglavlju prikazani su postignuti rezultati i njihova implementacija, Peto poglavlje prikazuje funkcionalnosti razvijenog sustava, dok se u zaključnom poglavlju daju smjernice za budući razvoj sustava.

2. PREGLED PODRUČJA TEME

Elektroničko vođenje sjednica i elektroničko glasanje postaju sve više zastupljeniji u suvremenim i multikulturnim organizacijama, upravnim tijelima i institucijama. Uvođenjem takvih sustava olakšava se i ubrzava proces donošenja odluka i praćenja istih sudjelovanjem svih članova organizacije.

Na tržištu postoji niz komercijalnih rješenja koja omogućuju provođenje e-sjednica i elektroničkog glasanja. Za pružanje funkcionalnosti poput stvaranja dnevnog reda, elektroničkog glasanja, bilježenja rezultata i izrade zapisnika sjednica jedni od poznatijih sustava su „Simply Voting“, „BigPulse“ i „MeetingBooster“. Brojne organizacije, također, koriste alate za videokonferencije poput „Zoom“, „Microsoft Teams“ ili „Google Meet“ kao osnovu za vođenje sjednice, no njihova primarna uporaba ne uključuje upravljanje glasanjem i prikazom rezultata te stvaranje brojnih točaka rasprave.

Ključne karakteristike e-sjednice su:

- Upravljanje sudionicima i njihovim ulogama
- Definiranje dnevnih točaka reda
- Glasanje
- Automatsko generiranje zapisnika
- Real-time notifikacije sudionicima
- Upravljanje dokumentacijom i priložima

Ovaj diplomski rad razvija vlastiti sustav koji objedinjuje ključne funkcionalnosti potrebne za provođenje e-sjednica, s naglaskom na glasanje u stvarnom vremenu, upravljanje korisničkim ulogama i sigurnost sustava, kao i na mogućnost prilaganja dokumenata različitih formata po točkama dnevnog reda te grafički prikaz konačnih rezultata glasanja. Za razliku od drugih komercijalnih rješenja, razvijeni sustav omogućava potpunu kontrolu nad arhitekturom, podacima i daljnjim nadogradnjama sustava, što je posebno važno za sustave koji se razvijaju unutar organizacija sa specifičnim zahtjevima i potrebama

3. OPIS SUSTAVA

3.1. Arhitektura web aplikacije

Web-aplikacija temelji se na klijent–server arhitekturi. Frontend je zadužen za korisničko sučelje i omogućuje interakciju s korisnikom, dok je backend odgovoran za poslovnu logiku, upravljanje podacima i obradu zahtjeva pristiglih s klijentske strane. Unutar aplikacije koriste se i WebSocket komunikacija, koja osigurava unutar stvarnog vremena održavanje glasanja svim korisnicima aplikacije. Za potrebe pohranjivanja podataka korištena je PostgreSQL baza podataka unutar koje su spremljeni svi podaci o korisnicima, stvorenim sjednicama i konkretnim točkama dnevnog reda te podacima vezanim za glasanje. Dokumenti koji se prilažu uz pojedine točke dnevnog reda spremaju se lokalno, a u bazi podataka evidentira se isključivo njihova putanja.

3.2. Korištene tehnologije

Razvoj web aplikacije podijeljen je na dva dijela, backend dio aplikacije i frontend dio aplikacije. Za backend dio aplikacije koristio se programski jezik Java, Spring Boot okvir koji omogućuje brzu i efikasnu izradu REST servisa. Spring ekosustav nudi razne module poput Spring Web modula za obradu HTTP zahtjeva, Spring Security modula za implementaciju sigurnosnih mehanizama te Spring Data JPA koji se koristi pri pristupanju bazi podataka i rada s entitetima. Cjelokupno upravljanje projektom i njegovim ovisnostima ostvareno je pomoću Maven alata, dok su podaci pohranjeni unutar PostgreSQL relacijske baze podataka [1]. Za potrebe autentifikacije i autorizacije korisnika koristi se JWT (JSON Web Token) standard koji omogućuje sigurno korištenje tokena [2]. Za potrebe pokretanja glasanja u stvarnom vremenu korištena je WebSocket komunikacija uz STOMP protokol i SockJS za postizanje kompatibilnosti s većinom preglednika.

Frontend dio aplikacije temeljen je na ReactJS biblioteci koja omogućuje modularan pristup razvijanja korisničkog sučelja. Upravljanje stanjima aplikacije postignuto je korištenjem MobX, koji omogućuje kontinuirano ažuriranje prikaza temeljem promjena u podatkovnom sloju. Za komunikaciju s backend dijelom korištena je Axios biblioteka koja pruža olakšan rad s REST API zahtjevima. Korištenjem STOMP over SockJS protokola postiže se jednostavna integracija WebSocket komunikacije unutar React aplikacija [3].

3.3. Opis poslužiteljske strane

Backend sustav razvijen je u programskom jeziku Java 17 uz primjenu Spring Boot okvira, koji omogućuje bržu i strukturiraniju implementaciju aplikacijske logike te pruža efikasnu podršku pri razvoju REST API servisa. Arhitektura je podijeljena u brojne slojeve kao što su: kontroleri, servisi, repozitoriji, entiteti i DTO modeli čime se u strukturi *koda* postiže veća čitljivost i mogućnost jednostavnijeg održavanje cijele strukture.

3.3.1. Entiteti baze podataka

Podaci aplikacije pohranjeni su u PostgreSQL relacijsku bazu podataka, pri čemu je svaki entitet definiran kao klasa označena anotacijom „@Entity“ i povezana s odgovarajućom tablicom. Glavni entiteti sustava su „korisnici“, „uloge“, „sjednice“, „točke“, „prilozi“ i „glasovi“. Entitet korisnici sadrži osnovne podatke o svakom korisniku, dok entitet uloge određuje tip korisničke uloge. Sjednice predstavljaju jednu elektroničku sjednicu s pripadajućim opisnim podacima, a putem entiteta točke omogućeno je dodavanje više točaka dnevnog reda unutar sjednice. Prilozi omogućuju spremanje dokumenata uz pojedinu točku i njihovu dostupnost svim korisnicima, dok entitet glasovi služi za evidentiranje rezultata održanoga glasanja po svakoj točki. Entiteti su međusobno povezani relacijama tipa „jedan-naprema-više“ (@OneToMany) i „više-naprema-jedan“ (@ManyToOne), što omogućuje navigaciju kroz podatkovni model i automatsko generiranje SQL upita pomoću Spring Data JPA [4].

3.3.2. Autentifikacija i autorizacija

Unutar web aplikacije korišten je JWT (JSON Web Token) koji predstavlja mehanizam za autentifikaciju korisnika. Nakon uspješne prijave, korisniku se ovisno o njegovim podacima, generira jedinstveni token. Token se proslijeđuje u svakom budućem zahtjevu unutar zaglavlja „Authorization : Bearer <token>“. Njegova uloga je da se na backend-u prilikom primitka API poziva validira korisnik pomoću JwtAuthenticationFilter klase. Autorizacija je implementirana koristeći Spring Security mehanizam te se korisnicima kontrolira pristup ovisno o njihovoj ulozi. Uloge se automatski prefiksiraju s „ROLE_<ime>“ što je nužno kako bi ih Spring Security ispravno prepoznao [5].

3.3.3. Servisni sloj

Svaka funkcionalnost vezana za rad s entitetima implementirana je u servisnim klasama, dok se za pristup bazi podataka i izvođenje konkretnih izmjena koristi pripadajući repozitorij. Sve provjere, obrade unesenih podataka i konverzacije potrebne za uspješno pohranjivanje obavljaju se u servisnom sloju. Korištenjem servisa postiže se veća razina organizacije unutar *koda*, mogućnost ponovnog korištenja logike na više različitih mjesta te najbitnije, testabilnost samoga *koda*.

3.3.4. WebSocket komunikacija

Implementiranjem dvosmjerne komunikacije putem WebSocket protokola unutar web aplikacije omogućeno je ažuriranje informacija potrebnih za glasanje u stvarnom vremenu. Za postizanje kompatibilnosti s velikom većinom preglednika koristi se STOMP protokol sa SockJS bibliotekom. Kako bi se to postiglo potrebno je definirati rute na koje se korisnici mogu preplatiti te putem kojih primaju poruke o statusu glasanja. U trenutku kada administrator promijeni status glasanja, svim pretplaćenim korisnicima šalje se poruka o spomenutoj promjeni za glasanje. Time se postiže automatsko ažuriranje stranice prije i poslije glasanja.

3.4. Opis korisničkog sučelja

Frontend dio aplikacije razvijen je korištenjem ReactJS biblioteke koja je odgovorna za izradu responzivnih korisničkih sučelja uz mogućnost prilagodljivosti i nadogradivosti. Razlog zbog kojeg je React korišten, u prvom redu su širi spektar performansi i fleksibilnosti koje nudi, kao i velik broj konkretnih gotovih rješenja i komponenti. Koristeći biblioteku MobX kompletna logika upravljanja stanjem odvaja se u posebne store klase.

3.4.1. Upravljanje stanjem pomoću MobX-a

MobX omogućuje globalno upravljanje stanjima unutar aplikacije na jednostavan i reaktivan način. Unutar MobX-ovog zasebnog store-a koji se koristi za svaki entitet pojedinačno nalaze se „observable“ varijable i zasebne metode koje su korištene dalje u *kodu* za dohvaćanje, ažuriranje i brisanje podataka. Unutar svakoga store-a korištenjem „makeAutoObservable“ funkcije automatski označavamo sve članove klase kao „observable“, „action“ ili „computed“ ovisno o njihovom tipu i načinu korištenja. Time se svim komponentama omogućuje automatska mogućnost renderiranja ukoliko dođe do promjene vrijednosti [6].

3.4.2. REST API komunikacija

Backend pruža određene RESTful API endpointove koji vraćaju podatke u JSON formatu. Za komunikaciju frontend i backend dijela aplikacije koristi se biblioteka Axios. Ona omogućuje jednostavnije slanje HTTP zahtjeva s React dijelom aplikacije na backend. Unutar svakog store-a nalazi se Service klasa gdje su definirane metode za slanje GET, PUT, POST, DELETE zahtjeva prema backend API-ju. Zahtjevi također sadržavaju JWT token koji je potrebno dodati u zaglavlje svakog zahtjeva za autentifikaciju [7].

3.4.3. WebSocket integracija

Za potrebe glasanja u stvarnom vremenu frontend aplikacija koristi protokol za povezivanje s backend WebSocket serverom. Potrebno je uspostaviti vezu prema zadanom endpointu s backend dijelom aplikacije te se pretplatiti na određene kanale s klijentske strane. Na taj način kada se pokrene glasanje za određenu točku, backend pošalje poruku svima koji su pretplaćeni na kanal i rezultira prikazivanjem modala za glasanje. Završetkom glasanja zatvara se modal i prikazuju se rezultati glasanja u stvarnom vremenu.

3.4.4. Ključne komponente korisničkog sučelja

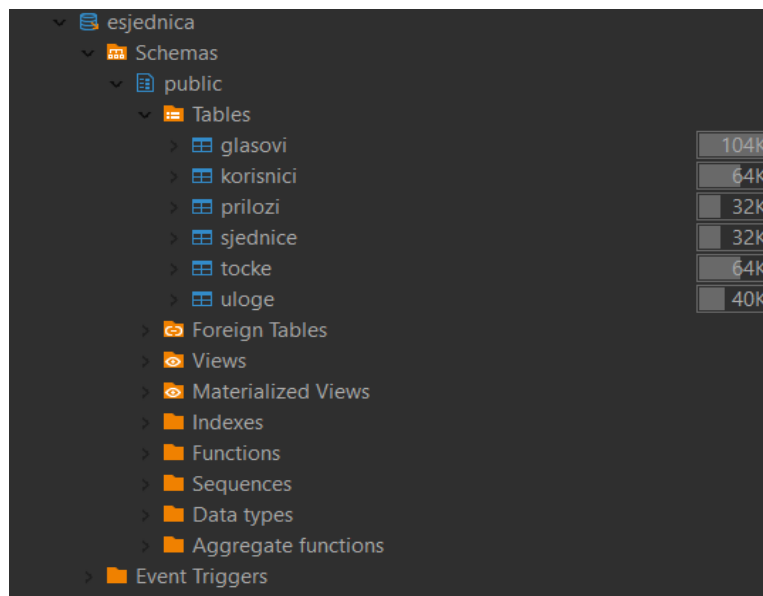
Frontend aplikacija sastoji se od brojnih stranica i komponenti koje zajedno predstavljaju i omogućuju rad e-sjednice. Neke od osnovnih stranica su stranice za prijavu, početna stranica, stranice unesenih sjednica i točki dnevnog reda. Korisnik prilikom ulaska na stranicu ima pogled na početnu stranicu nakon čega je potrebna prijava kako bi pristupio ostalom dijelu aplikacije. Ovisno u korisnikovoj ulozi postavljena su razna ograničenja.

4. REZULTATI IMPLEMENTACIJE

Završetkom implementacije web aplikacije napravljeno je funkcionalno testiranje osnovnih dijelova aplikacije kako bi se utvrdilo njihovo očekivano ponašanje. U nastavku su opisani konačni rezultati i ispitane funkcionalnosti. Implementacija sustava podijeljena je u četiri cjeline: baza podataka koja definira entitete i njihove relacije, backend razvijen u Javi sa Spring Bootom sadrži poslovnu logiku i REST API, frontend u Reactu omogućuje interakciju s korisnikom, dok je dizajn stranice usmjeren na preglednost i jednostavnost korištenja.

4.1. Implementacija baze podataka

Struktura baze podataka prikazana je na slici 4.1, kao što je prikazano na slici unutar baze podataka nalaze se tablice „korisnici“, „uloge“, „sjednice“, „tocke“, „glasovi“ i „prilozi“.



Slika 4.1: Struktura baze podataka

Tablica uloge predstavlja „lookup“ tablicu koja sadrži podatke o dostupnim ulogama koje se koriste na stranici. Njezina struktura prikazana je na slici 4.2.

```
CREATE TABLE public.uloze (
    id serial4 NOT NULL,
    naziv varchar(20) NOT NULL,
    CONSTRAINT uloze_naziv_key UNIQUE (naziv),
    CONSTRAINT uloze_pkey PRIMARY KEY (id)
);
```

Slika 4.2: Tablica uloga

Tablica „korisnici“ sadrži sve podatke vezane za prijavljenog korisnika, prikazano na slici 4.3. Unutar tablice primarni ključ je korisnikov id, dok su za korisničko ime i email postavljena ograničenja da njihova vrijednost mora biti jedinstvena.

```
CREATE TABLE public.korisnici (
    id bigserial NOT NULL,
    ime varchar(50) NOT NULL,
    prezime varchar(50) NOT NULL,
    email varchar(100) NOT NULL,
    lozinka varchar(255) NOT NULL,
    roles varchar(255) NULL,
    username varchar(255) NOT NULL,
    CONSTRAINT korisnici_email_key UNIQUE (email),
    CONSTRAINT korisnici_pkey PRIMARY KEY (id),
    CONSTRAINT uk_fnhlgpdv9nwggcetnb4hadcdi UNIQUE (username)
);
```

Slika 4.3: Tablica korisnici

Tablica „sjednice“ sadrži podatke o konkretnoj sjednici koja je unesena. Id sjednice predstavlja primarni ključ, dok se sekundarni ključ odnosi na id korisnika koji ju je stvorio. Ukoliko se korisnikov id obriše, unutar sjednica se njegova vrijednost postavlja na NULL. Također se pohranjuju ostali ključni podaci, kao što su lokacija održavanja sjednice s datumom održavanja i datumom stvaranja sjednice.

```
CREATE TABLE public.sjednice (
    id bigserial NOT NULL,
    naziv varchar(255) NOT NULL,
    opis text NULL,
    datum_odrzavanja timestamp(6) NOT NULL,
    kreirao_id int4 NULL,
    datum date NOT NULL,
    lokacija varchar(255) NULL,
    CONSTRAINT sjednice_pkey PRIMARY KEY (id),
    CONSTRAINT sjednice_kreirao_id_fkey FOREIGN KEY (kreirao_id) REFERENCES public.korisnici(id) ON DELETE SET NULL
);
```

Slika 4.4: Tablica sjednice

Tablica „tocke“ odnosi se na podatke vezane za dnevnu točku reda koja se unosi unutar sjednica, prikazano na slici 4.5. Unutar tablice primarni ključ je id točke, dok sekundarni ključ ima vrijednost id sjednice unutar koje se konkretna točka dnevnog reda nalazi. Također, unutar tablice nalaze se podaci o tome kada je glasanje započelo te koliko traje.

```
CREATE TABLE public.tocke (  
  id bigserial NOT NULL,  
  sjednica_id int8 NULL,  
  naziv varchar(255) NOT NULL,  
  opis text NULL,  
  glasanje_start timestamp NULL,  
  glasanje_trajanje int4 NULL,  
  CONSTRAINT tocke_pkey PRIMARY KEY (id),  
  CONSTRAINT tocke_sjednica_id_fkey FOREIGN KEY (sjednica_id) REFERENCES public.sjednice(id) ON DELETE CASCADE  
);
```

Slika 4.5: Tablica tocke

Tablica „glasovi“ odnosi se na podatke o glasanju koje se odvijalo na točkama dnevnog reda, prikazano na slici 4.6. Unutar tablice nalaze se podaci o korisniku koji je glasao te za koju je točku dnevnog reda glas namijenjen. Postavljena su ograničenja, tako da za istu točku dnevnog reda, isti korisnik ne može više puta glasati. Primarni ključ tablice čini atribut id glasa, dok su sekundarni ključevi identifikatori korisnika i točke dnevnog reda s kojom je glas povezan.

```
CREATE TABLE public.glasovi (  
  id bigserial NOT NULL,  
  korisnik_id int8 NULL,  
  tocka_id int8 NULL,  
  glas varchar(255) NOT NULL,  
  CONSTRAINT glasovi_korisnik_id_tocka_id_key UNIQUE (korisnik_id, tocka_id),  
  CONSTRAINT glasovi_pkey PRIMARY KEY (id),  
  CONSTRAINT ukq4d6noothaobvgyyqi3lxxol UNIQUE (korisnik_id, tocka_id),  
  CONSTRAINT unique_korisnik_tocka UNIQUE (korisnik_id, tocka_id),  
  CONSTRAINT glasovi_korisnik_id_fkey FOREIGN KEY (korisnik_id) REFERENCES public.korisnici(id) ON DELETE CASCADE,  
  CONSTRAINT glasovi_tocka_id_fkey FOREIGN KEY (tocka_id) REFERENCES public.tocke(id) ON DELETE CASCADE  
);
```

Slika 4.6: Tablica glasovi

Tablica „prilozi“ predstavlja podatke o dodanim prilogima na svakoj točki dnevnog reda, prikazano na slici 4.7. Sadrži glavne podatke ključne za daljnje korištenje kako bi ostali korisnici mogli preuzeti priložene dokumente. Primarni ključ u entitetu priloga čini atribut id, dok je sekundarni ključ vrijednost id točke s kojom je prilog povezan. U bazi podataka pohranjuje se samo putanja do konkretnog dokumenta, dok se sam dokument sprema lokalno.

```
CREATE TABLE public.prilozi (  
  id bigserial NOT NULL,  
  naziv varchar(255) NULL,  
  ekstenzija varchar(20) NULL,  
  tocka_id int8 NULL,  
  putanja varchar(500) NULL,  
  CONSTRAINT prilozi_pkey PRIMARY KEY (id),  
  CONSTRAINT prilozi_tocka_id_fkey FOREIGN KEY (tocka_id) REFERENCES public.tocke(id) ON DELETE CASCADE  
);
```

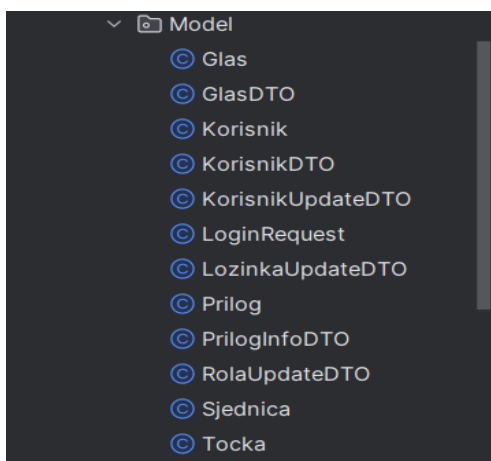
Slika 4.7: Tablica prilozi

4.2. Implementacija Backend dijela aplikacije

Backend aplikacija razvijena je u programskom jeziku Java uz korištenje Spring Boot okvira i organizirana prema višeslojnoj arhitekturi. Kontrolni sloj obrađuje zahtjeve i komunicira s korisničkim sučeljem putem REST API-ja, dok servisni sloj sadrži poslovnu logiku. Repozitorski sloj upravlja pristupom bazi podataka, sloj modela definira entitete i njihove relacije, a sigurnosni sloj osigurava autentifikaciju, autorizaciju i zaštitu podataka.

4.2.1. Sloj modela

Unutar sloja za modele nalaze se svi entiteti iz baze podataka kako bi se njihove vrijednosti pohranile unutar aplikacije za daljnje procesuiranje. Svaki entitet nalazi se u zasebnoj klasi prema strukturi prikazanoj na slici 4.8.



Slika 4.8: Struktura sloja za modele

Unutar sloja nalaze se i pripadajući DTO modeli koji služe kao povratni tipovi za konkretne entitete iz baze, oni sadrže samo podatke koji su potrebni za njihov prikaz na web aplikaciji. Koriste se prilikom slanja podataka između slojeva aplikacije ili vanjskog izvora bez izlaganja nepotrebnih ili povjerljivih podataka. Primjer korištenja DTO modela prikazan je na slici 4.9, dok je njezin originalni entitet prikazan na slici 4.10.

```
© GlasDTO.java x © Glas.java
1 package com.example.esjednica.Model;
2
3 public class GlasDTO { 2 usages 1 Lsasko
4     private String glas; 2 usages
5
6     public String getGlas() { return glas; }
7
8
9
10    public void setGlas(String glas) { this.glas = glas; }
11
12
13 }
14
```

Slika 4.9: DTO model za entitet Glas

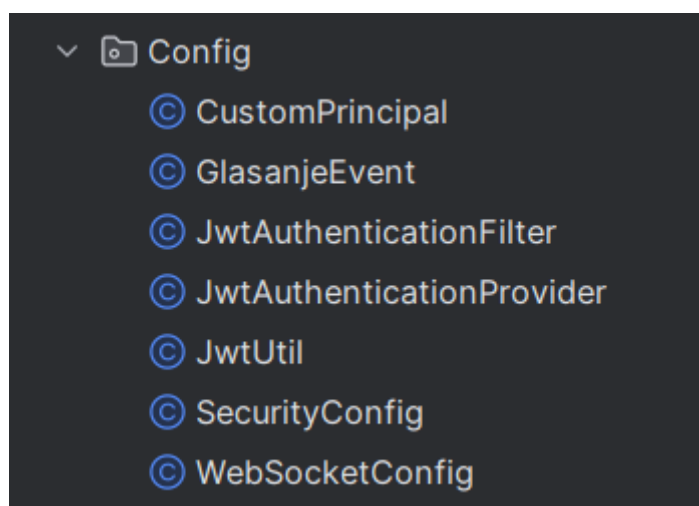
```
© GlasDTO.java © Glas.java x
1 package com.example.esjednica.Model;
2
3 import jakarta.persistence.*;
4
5 @Entity 1 Lsasko
6 @Table(name = "glasovi", uniqueConstraints = {@UniqueConstraint(columnNames = {"korisnik_id", "tocka_id"})})
7 public class Glas {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @Column(name = "korisnik_id") 2 usages
14    private Long korisnikId;
15
16    @Column(name = "tocka_id") 2 usages
17    private Long tockaId;
18
19    @Column(nullable = false) 2 usages
20    private String glas;
21
22    // Getteri i setteri
23    public Long getId() { return id; } 1 Lsasko
24    public void setId(Long id) { this.id = id; } 1 Lsasko
25    public Long getKorisnikId() { return korisnikId; } no usages 1 Lsasko
26    public void setKorisnikId(Long korisnikId) { this.korisnikId = korisnikId; } 1 usage 1 Lsasko
27    public Long getTockaId() { return tockaId; } 1 Lsasko
28    public void setTockaId(Long tockaId) { this.tockaId = tockaId; } 1 Lsasko
29    public String getGlas() { return glas; } 2 usages 1 Lsasko
30    public void setGlas(String glas) { this.glas = glas; } 1 usage 1 Lsasko
31 }
```

Slika 4.10: Glas entitet

Svi entiteti u aplikaciji označeni su anotacijom „@Entity“, čime Spring Data JPA klasu mapira na stvarnu tablicu u bazi podataka. „@Table“ anotacija omogućuje eksplicitno navođenje naziva tablice, što je nužno ukoliko se ime modela razlikuje od imena tablice unutar baze podataka. „@Id“ i „@GeneratedValue“ definiraju primarni ključ koji se automatski generira. „@Column“ se koristi za mapiranje atributa na stupce u bazi te omogućavanje dodatnih postavki kao što su „nullable“, „unique“, „length“ i slično. Svaki atribut unutar modela ima pridružene „gettere“ i „settere“, što omogućava čitanje i pisanje njegove vrijednosti. Ovim pristupnim metodama frameworku se omogućuje automatska serializacija, deserializacija i mapiranje podataka.

4.2.2. Sigurnosni sloj

Sigurnosni sloj aplikacije služi za definiranje sigurnosnih pravila, pristupnih tokova i postavki WebSocketa. Temeljen je na JWT autentifikaciji, čime je osigurano da samo registrirani korisnici pristupaju zaštićenim resursima, odnosno podacima. Struktura sloja prikazana je na slici 4.11.



Slika 4.11: Sigurnosni sloj

Unutar strukture backend sustava nalaze se klase zadužene za autentifikaciju korisnika putem JWT tokena, zaštitu i autorizaciju endpointova, konfiguraciju WebSocketa te definiranje korisničkog konteksta. Klasa „SecurityConfig“ određuje koje su rute javno dostupne, a koje zahtijevaju zaštitu, te omogućuje rad „JwtAuthenticationFiltera“. Taj filter presreće svaki HTTP zahtjev i provjerava valjanost JWT tokena. Ako je token ispravan, korisnikov identitet se proslijeđuje dalje kroz „SecurityConfig“. Na slici 4.12 prikazana je konfiguracija sigurnosti, dok je primjena filtera prikazana na slici 4.13.

```

public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(CorsConfigurer<HttpSecurity> cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
            .authorizeHttpRequests(AuthorizationManagerRequestMatcherConfigurer::authorize -> authorize
                .requestMatchers("/api/users/**").permitAll()
                .requestMatchers("/api/glasanja/**").authenticated()
                .requestMatchers("/api/sjednice/**").authenticated()
                .requestMatchers("/api/tocke/**").authenticated()
                .requestMatchers("/ws/**").permitAll()
                .anyRequest().permitAll()
            )
            .authenticationProvider(jwtAuthenticationProvider)
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

Slika 4.12: Autorizacija endpointova unutar SecurityConfig klase

```

19 public class JwtAuthenticationFilter extends OncePerRequestFilter {
30     protected void doFilterInternal(HttpServletRequest request,
40         HttpServletResponse response) throws ServletException, IOException {
41         String token = request.getHeader("Authorization").substring(7);
42         String username = jwtUtil.extractUsername(token);
43
44         if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
45             UserDetails userDetails = userService.loadUserByUsername(username);
46
47             if (jwtUtil.isTokenValid(token)) {
48                 var authorities = jwtUtil.extractRoles(token).stream().map(role -> role.startsWith("ROLE_") ? role : "ROLE_" + role.toUpperCase())
49                     .map(SimpleGrantedAuthority::new).collect(Collectors.toList());
50
51                 Long userId = jwtUtil.extractUserId(token);
52                 CustomPrincipal principal = new CustomPrincipal(userId, username);
53                 UsernamePasswordAuthenticationToken auth =
54                     new UsernamePasswordAuthenticationToken(principal, null, authorities);
55
56                 SecurityContextHolder.getContext().setAuthentication(auth);
57             }
58         }
59     }
60 }

```

Slika 4.13: Provjera tokena unutar filtera JwtAuthenticationFilter

Klasa „*JwtAuthenticationProvider*“ definira način validacije korisnika na temelju JWT tokena te u suradnji s klasama „*CustomPrincipal*“ i „*JwtUtil*“ vraća validiranog korisnika s pripadajućom ulogom. Klasa „*JwtUtil*“ služi za rad s JWT tokenima – generira ih prilikom prijave te parsira potrebne podatke, dok „*CustomPrincipal*“ predstavlja prilagođenu sigurnosnu implementaciju korisnika. Ona iz tokena preuzima i pohranjuje korisničke podatke koji se zatim koriste za autorizaciju i pristup unutar aplikacije nakon uspješne autentifikacije. Primjena navedenih klasa („*provider*“, „*JwtUtil*“ i „*CustomPrincipal*“) prikazana je na slikama 4.14, 4.15 i 4.16

```

13 public class JwtAuthenticationProvider implements AuthenticationProvider {
14
15     @Override
16     public Authentication authenticate(Authentication authentication) throws AuthenticationException {
17         String token = (String) authentication.getCredentials();
18         if (token == null || !token.startsWith("Bearer ")) {
19             throw new BadCredentialsException("Invalid token format");
20         }
21
22         token = token.substring(7);
23         String username = jwtUtil.extractUsername(token);
24
25         if (username != null) {
26             UserDetails userDetails = userDetailsService.loadUserByUsername(username);
27             if (jwtUtil.isTokenValid(token)) {
28                 return new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
29             }
30         }
31
32         throw new BadCredentialsException("Invalid or expired token");
33     }
34 }

```

Slika 4.14: „JwtAuthenticationProvider“ klasa

```

14 public class JwtUtil {
15
16     // Generiranje tokena s ID-em i ulogama
17     public String generateToken(String username, List<String> roles, Long userId) {
18         return Jwts.builder()
19             .setSubject(username)
20             .claim("roles", roles)
21             .claim("id", userId)
22             .setIssuedAt(new Date())
23             .setExpiration(new Date(System.currentTimeMillis() + expirationTimeInMs))
24             .signWith(signingKey, SignatureAlgorithm.HS256)
25             .compact();
26     }
27
28     // Provjera valjanosti tokena
29     public static boolean isTokenValid(String token) {
30         try {
31             extractAllClaims(token);
32             return true;
33         } catch (JwtException | IllegalArgumentException e) {
34             return false;
35         }
36     }
37
38     // Dohvat korisničkog imena
39     public String extractUsername(String token) {
40         return extractAllClaims(token).getSubject();
41     }
42
43     // Dohvat korisničkog ID-a
44     public static Long extractUserId(String token) {
45         return extractAllClaims(token).get("id", Long.class);
46     }
47
48     // Dohvat korisničkih uloga
49     public List<String> extractRoles(String token) {
50         return extractAllClaims(token).get("roles", List.class);
51     }
52
53     // Sve claimove
54     private static Claims extractAllClaims(String token) {
55         return Jwts.parserBuilder()
56             .setSigningKey(signingKey)
57             .build()
58             .parseClaimsJws(token)
59             .getBody();
60     }
61 }

```

Slika 4.15: JwtUtil klasa


```

1 package com.example.esjednica.Config;
2
3 public class CustomPrincipal { 5 usages  Lsasko
4     private final Long id; 2 usages
5     private final String username; 2 usages
6
7     public CustomPrincipal(Long id, String username) { 1 usage  Lsasko
8         this.id = id;
9         this.username = username;
10    }
11
12    > public Long getId() { return id; }
15
16    > public String getUsername() { return username; }
19 }
20

```

Slika 4.16: CustomPrincipal klasa

4.2.3. Kontrolni sloj

Kontrolni sloj je zadužen za prihvatanje HTTP zahtjeva i njihovo proslijeđivanje za daljnje korištenje unutar servisnog sloja. Popis svih kontrolera aplikacije prikazan je na slici 4.17.



Slika 4.17: Kontrolni sloj

Svaka tablica unutar baze podataka, osim „lookup“ tablice uloga ima svoj konkretni kontroler koji je zadužen za zaprimanje konkretnih HTTP zahtjeva vezanih za njih. Unutar kontrolnog sloja provjeravaju se ulazni podaci koji se šalju kao parametri te se proslijeđuju na daljnju obradu servisnom sloju. Najčešće kontrolni sloj obrađuje CRUD zahtjeve koji su ključni za stvaranje, čitanje, ažuriranje i brisanje podataka unutar baze podataka. Jedna klasa kontrolera može koristiti više različitih klasi servisnog sloja unutar sebe. „UserController“ klasa odgovorna je za upravljanje zahtjevima za prijavu i registraciju novih i postojećih korisnika.

Metoda za registraciju novih korisnika prikazana je na slici 4.18. Osim toga, koristi se za obradu zahtjeva za ažuriranje korisnikovog profila, promjenom lozinke te promjenom korisničke uloge. Samo korisnici s ulogama administratora i predlagatelja mogu uređivati drugim postojećim korisnicima njihove uloge. Većina metoda u kontrolnom sloju vrši provjeru korisnikove uloge s tokena kako bi se ograničilo pravo pristupa određenim metodama. Ukoliko su metode javno dostupne svima u aplikaciji, nije postavljena autorizacija.

```
@PostMapping("/register")
public ResponseEntity<Korisnik> registerUser(@RequestBody Korisnik user) {
    String requestedRoles = user.getRoles();
    if (requestedRoles == null || requestedRoles.isEmpty()) {
        throw new IllegalArgumentException("Roles cannot be empty");
    }

    Set<String> rolesToCheck = new HashSet<>(Arrays.asList(requestedRoles.split(", ")));
    if (!ALLOWED_ROLES.containsAll(rolesToCheck)) {
        throw new IllegalArgumentException("Invalid or unauthorized role(s) provided. Allowed roles: " + ALLOWED_R
    }

    user.setLozinka(passwordEncoder.encode(user.getLozinka()));
    Korisnik savedUser = userRepository.save(user);
    return ResponseEntity.ok(savedUser);
}
```

Slika 4.18: Metoda za registraciju korisnika

Kao što je vidljivo na slici 4.18, metoda kao parametre prima objekt klase Korisnik. Dobiveni podaci se provjeravaju te ukoliko nedostaju neki od potrebnih podataka vraća se iznimka. Nakon toga se vrši provjera unesene uloge s listom dozvoljenih uloga što sprječava stvaranje korisnika s nedozvoljenim ulogama. Metoda za prijavu postojećih korisnika prikazana je na slici 4.19.

```

@PostMapping("/login")
public ResponseEntity<?> loginUser(@RequestBody LoginRequest loginRequest) {
    Optional<Korisnik> optionalUser = userRepository.findByUsername(loginRequest.getUsername());
    if (optionalUser.isPresent()) {
        Korisnik user = optionalUser.get();
        if (passwordEncoder.matches(loginRequest.getLozinka(), user.getLozinka())) {
            List<String> rolesList = Arrays.asList(user.getRoles().split(" "));
            String token = jwtUtil.generateToken(user.getUsername(), rolesList, user.getId());
            Map<String, Object> response = new HashMap<>();
            response.put("token", token);
            Map<String, Object> userInfo = new HashMap<>();
            userInfo.put("id", user.getId());
            userInfo.put("username", user.getUsername());
            userInfo.put("roles", Arrays.asList(user.getRoles().split(" ")));
            userInfo.put("ime", user.getIme());
            userInfo.put("prezime", user.getPrezime());
            userInfo.put("email", user.getEmail());
            response.put("user", userInfo);
            return ResponseEntity.ok(response);
        }
    }
    return ResponseEntity.status(401).body("Invalid credentials");
}

```

Slika 4.19: Metoda za prijavu korisnika

Metoda kao parametar prima klasu „loginRequest“ s unesenim podacima za korisnikovo ime i lozinku iz prijavnog obrasca. Unutar aplikacije prilikom prijave pokušava se pronaći korisnik prema unesenom korisničkom imenu, provjerava se podudaranje unesene lozinke s pohranjenom lozinkom iz baze podataka. U slučaju da neka od provjera ne uspije kao rezultat se vraća greška o pogrešno unesenim podacima. Nakon uspješne provjere unesenih podataka generira se token za trenutno prijavljenog korisnika, popunjavaju se potrebni korisnički podaci te se isti vraćaju kao rezultat. Ostali kontroleri koriste se za obradu zahtjeva za stvaranje, brisanje, ažuriranje i čitanje entiteta iz baze podataka. Unutar njih provjeravaju se podaci za konkretnu klasu modela te se vrši validacija tih podataka kako bi se spriječilo unošenje neispravnih ili nepotpunih informacija. Kontroleri koriste svoje pripadajuće servisne klase, dok kontroler za glasanje koristi i servisnu klasu za točku dnevnog reda. Prilikom deklariranja kontrolera potrebno je anotirati klasu s „@RestController“ te postaviti željenu rutu za taj zahtjev, kao što je vidljivo na slici 4.20.

```

24
25
26 @RestController  Lsasko
27 @RequestMapping("/api/glasanje")
28 public class GlasanjeController {
29     @Autowired
30     private GlasService glasService;
31     @Autowired
32     private SimpMessagingTemplate messagingTemplate;
33     @Autowired
34     private TockaService tockaService;
35
36

```

Slika 4.20: Korištene servisne klase unutar kontrolera za glasanje

```

@PostMapping("/tocka/{tockId}")  Lsasko
@PreAuthorize("hasAnyRole('ADMIN', 'KORISNIK', 'GLEDATELJ', 'PREDLAGATELJ')")
public ResponseEntity<String> glasaj(@PathVariable Long tockId, @RequestBody GlasDTO glasDTO, Authentication authentication) {

    CustomPrincipal user = (CustomPrincipal) authentication.getPrincipal();

    if(!tockService.isGlasanjeAktivno(tockId)) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body("Glasanje nije aktivno za ovu točku.");
    }
    if (glasService.hasUserAlreadyVoted(tockId, user.getId())) {
        return ResponseEntity.status(HttpStatus.CONFLICT).body("Korisnik je već glasao.");
    }

    Glas glas = new Glas();
    glas.setGlas(glasDTO.getGlas());
    glas.setKorisnikId(user.getId());
    glas.setTockId(tockId);
    return ResponseEntity.ok(glasService.saveGlas(glas).toString());
}

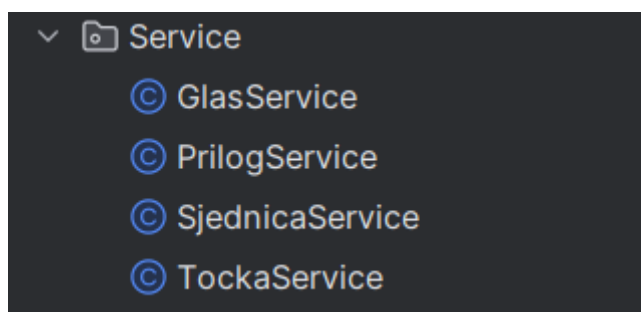
```

Slika 4.21: Primjer implementacije metode unutar kontrolera

Anotacijom „@PreAuthorize(“hasRole('ULOGA')”)“ prilikom implementiranja osnovnih metoda kontrolera ograničava se pristup određenim metodama isključivo korisnicima s pripadajućom ulogom. Na taj način provedena je autorizacija na razini pojedinog HTTP zahtjeva. Slika 4.21 prikazuje korištenje „PreAuthorize“ anotacije te korištenje dvije servisne klase unutar jednog HTTP zahtjeva. Prilikom glasanja potrebna je informacija iz servisne klase za točku dnevnog reda budući da se unutar nje nalazi podatak o aktivnosti glasanja. HTTP zahtjev prilikom poziva za evidentiranje glasa prvotno provjerava je li glasanje aktivno, nakon čega se vrši provjera za trenutno prijavljenog korisnika ukoliko je već glasao. Nakon tih provjera evidentira se poslani glas te je time onemogućeno evidentiranje glasanja koje nije aktivno, kao i mogućnost pojave dvostrukih glasova jednog korisnika.

4.2.4. Servisni sloj

Servisni sloj sadrži klase potrebne za obradu HTTP zahtjeva prije njihovog korištenja unutar repozitориjskog sloja. Obraduje podatke neposredno prije prosljeđivanja ili nakon dobivanja podataka s repozitориjskog sloja ovisno o vrsti zahtjeva. Unutar servisnog sloja postavljaju se i procesuiraju dodatni podaci koji nisu uneseni u kontrolnom sloju ili nisu dobiveni s baze podataka. To uključuje postavljanje vremenskih vrijednosti, formatiranje podataka, matematički izračuni i druge slične radnje. Također je moguća kombinacija korištenja većeg broja servisnih klasa ili prethodno implementiranih metoda za razne druge poslove. Unutar sloja se također vrši validacija podataka kako bi se izbjegle brojne pogreške. Servisni sloj zajedno s pripadajućim klasama prikazan je na slici 4.22.



Slika 4.22: Servisni sloj

Servisni sloj sadrži klase za četiri entiteta baze podataka čiji se podaci obrađuju prilikom HTTP zahtjeva. Na slici 4.23 prikazana je servisna klasa za priloge te se unutar te klase koristi pripadajuća repozitориjska klasa. Servisne klase označene su anotacijom „@Service“, čime se označavaju kao Spring servisne komponente i omogućava se njihovo automatsko upravljanje putem Springovog kontejnera ovisnosti. Prilozi se pohranjuju lokalno unutar „UPLOAD_DIR“ definirane datoteke.

```

@Service 2 usages 1 Lsasko *
public class PrilogService {

    private static final String UPLOAD_DIR = "uploads"; 1 usage

    @Autowired
    private PrilogRepository prilogRepository;

    public Prilog spremi(MultipartFile file, Long tockaId) throws IOException { 1 usage 1 Lsasko *
        String original = file.getOriginalFilename();
        String ocisceniNaziv = ocistiNaziv(original);
        String ekstenzija = ocisceniNaziv.substring(beginIndex: ocisceniNaziv.lastIndexOf(ch: '.') + 1);
        String noviNaziv = UUID.randomUUID() + "_" + ocisceniNaziv;
        Path path = Paths.get(UPLOAD_DIR, noviNaziv);
        Files.createDirectories(path.getParent());
        Files.write(path, file.getBytes());
        Prilog prilog = new Prilog();
        prilog.setNaziv(ocisceniNaziv);
        prilog.setEkstenzija(ekstenzija);
        prilog.setPutanja(path.toString());
        prilog.setTockaId(tockaId);
        return prilogRepository.save(prilog);
    }
}

```

Slika 4.23: Servisna klasa za priloge

Unutar servisne klase nalazi se metoda koja se koristi za obradu podataka prije njihovog pohranjivanja u bazu podataka. Prije samog pohranjivanja podataka potrebno je obraditi naziv same datoteke, odnosno očistiti njezin naziv, odrediti ekstenziju datoteke i generirati novi jedinstveni naziv pogodan za njezino spremanje. Za potrebe čišćenja naziva datoteke implementirana je metoda „ocistiNaziv“ prikazana na slici 4.24.

```

private String ocistiNaziv(String original) { 1 usage  ⚡ Lsasko *
    String ekstenzija = "";

    int dotIndex = original.lastIndexOf( ch: '.');
    if (dotIndex != -1) {
        ekstenzija = original.substring(dotIndex);
        original = original.substring(0, dotIndex);
    }

    String naziv = original
        .replaceAll( regex: "\\s+", replacement: "_")
        .replaceAll( regex: "[^a-zA-Z0-9_-]", replacement: "");

    return naziv + ekstenzija.toLowerCase();
}

```

Slika 4.24: Metoda za čišćenje naziva datoteke

Metoda uzima originalni naziv datoteke, odvaja ekstenziju datoteke od nazive, zatim zamjenjuje sve razmake sa znakom '_' te uklanja ostale znakove koji nisu slovo, broj, '-' ili '_'. Naziv se sprema bez velikih slova. Ostale korištene metode prikazane su na slici 4.25.

```

public Prilog getById(Long id) { 1 usage  ⚡ Lsasko
    return prilogRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Prilog s ID " + id + " ne postoji."));
}

public void obrisiPoId(Long id) throws IOException { 1 usage  ⚡ Lsasko
    Prilog prilog = prilogRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Prilog s ID " + id + " ne postoji."));

    Path path = Paths.get(prilog.getPutanja());
    Files.deleteIfExists(path);

    prilogRepository.deleteById(id);
}

public List<Prilog> dohvatiZaTocku(Long tockaId) { return prilogRepository.findByTockaId(tockaId); }

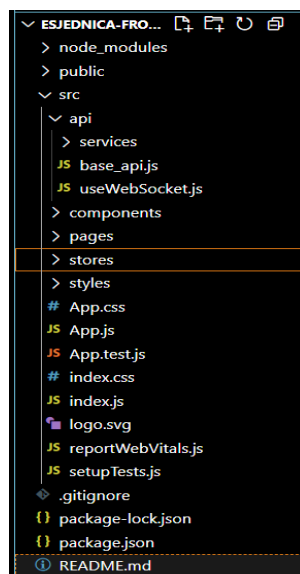
```

Slika 4.25: Ostale metode servisne klase za prilog

Postoje dvije metode za dohvaćanje priloga, metoda „getById“ dohvaća konkretan prilog po njegovom id, dok metoda „dohvatiZaTocku“ dohvaća sve priloge koji su povezani za konkretnu točku dnevnog reda. Zadnja metoda unutar klase koristi se za brisanje priloga, metoda osim što briše podatke s baze podataka, mora i obrisati lokalno pohranjenu datoteku.

4.3. Implementacija frontend dijela aplikacije

Frontend dio web aplikacije implementiran je koristeći ReactJs biblioteku, čime je omogućena izgradnja dinamičkih i responzivnih sučelja korištenjem više zasebnih komponenti. Glavni razlog odabira React-a je njegova fleksibilnost, jednostavnost upravljanja podataka i velika podrška zajednice. Na backend dio web aplikacije šalju se REST API zahtjevi koristeći JWT autentifikaciju za sigurnost. Nakon dobivenog odgovora korištena je biblioteka MobX za upravljanje stanjima unutar aplikacije. MobX omogućuje automatsku sinkronizaciju stanja, pojednostavljen prikaz podataka i njihov reaktivan pristup. Aplikacija podržava obavijesti u stvarnom vremenu korištenjem WebSocket komunikacije što omogućuje trenutno prikazivanje informacija o pokretanju i završetku glasanja svim prijavljenim korisnicima. Sve funkcionalnosti organizirane su unutar odvojenih React komponenti i pripadajućih „store“ klasa, čime se postiže modularnost, preglednost i jednostavno održavanje *koda*. U nastavku poglavlja detaljno su opisani glavni dijelovi frontend arhitekture, organizacija projektnog *koda*, upravljanje stanjem pomoću MobX-a, komunikacija s REST servisima, WebSocket integracija kao i funkcionalnosti korisničkog sučelja.



Slika 4.26: Struktura frontend aplikacije

4.3.1. Servisna klasa za stvaranje API poziva

Unutar aplikacije implementirana je servisna klasa koja koristi biblioteku Axios za slanje HTTP zahtjeva. Konkretno servisne klase nalaze se unutar api datoteke zajedno s klasom „base_api.js“ koja definira osnovne metode za komunikaciju s backend dijelom aplikacije, odnosno definira bazne metode koje se unutar konkretnih servisnih klasa koriste za komunikaciju.

Bazna klasa prilikom poziva zahtjeva nadodaje JWT token koji je pohranjen u lokalnoj memoriji, bazna klasa prikazana je na slici 4.27.

```
src > api > JS base_api.js > ...
1 import axios from 'axios';
2 const BASE_URL = 'http://localhost:8080/api';
3 const axiosInstance = axios.create({
4   |   baseURL: BASE_URL,
5   | });
6 axiosInstance.interceptors.request.use((config) => {
7   |   const token = localStorage.getItem('token');
8   |   if (token) {
9   |     | config.headers.Authorization = `Bearer ${token}`;
10  |   }
11  |   return config;
12  | });
13 export const get = (url) => axiosInstance.get(url);
14 export const post = (url, data) => axiosInstance.post(url, data);
15 export const put = (url, data) => axiosInstance.put(url, data);
16 export const del = (url) => axiosInstance.delete(url);
17 export const downloadFile = async (url) => {
```

Slika 4.27: Bazna servisna klasa

Svaka pojedinačna servisna klasa preuzima potrebne osnovne metode za stvaranje zahtjeva iz bazne klase. Unutar sebe također definira konstantu „BASE“ koja predstavlja potrebnu putanju za slanje specifičnog zahtjeva. Servisne klase stvaraju se za svaki konkretni kontroler s backend dijela aplikacije kako bi mogli komunicirali s njim. Primjer konkretne servisne klase za sjednicu prikazan je na slici 4.28.

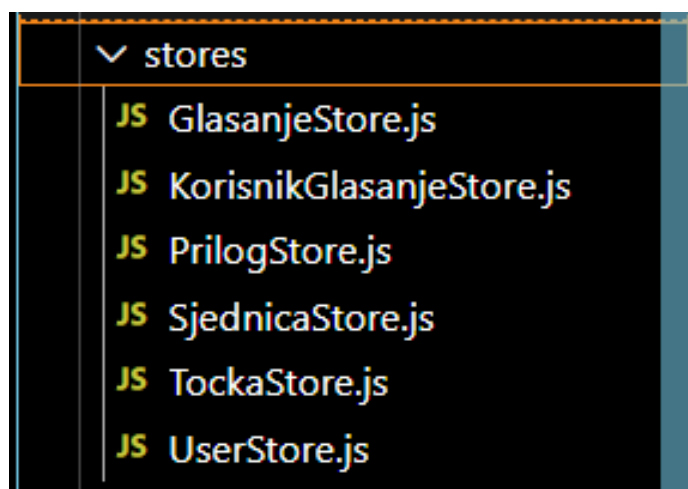
```
src > api > services > JS SjednicaService.js > ...
1 import { get, post, put, del } from '../base_api';
2 const BASE = '/sjednice';
3
4 export const SjednicaService = {
5   |   getAll: () => get(BASE),
6   |   getById: (id) => get(`${BASE}/${id}`),
7   |   create: (data) => post(BASE, data),
8   |   update: (id, data) => put(`${BASE}/${id}`, data),
9   |   delete: (id) => del(`${BASE}/${id}`)
10  | };
11
```

Slika 4.28: Servisna klasa za sjednicu

4.3.2. Upravljanje stanjima pomoću MobX store

Primjenom MobX store klasa postignuta je jasna podjela odgovornosti između prikaza i poslovne logike, što rezultira povećanom održivosti i preglednosti frontend-a. Unutar svake klase nalaze se „observable“ varijable specifične za vrstu podataka čijim stanjima upravljaju. Ključne mehanizme koje možemo izdvojiti korištenjem MobX-a su „observable“, „action“ i „computed“.

„Observable“ označava varijable koje predstavljaju stanje i koji MobX automatski prati. Promjena tih stanja odvija se korištenjem metoda koji su označene kao „action“, dok se „computed“ koristi za nekakve izvedene vrijednosti koji ovise o „observable“ varijablama. Unutar aplikacije korištena je „makeAutoObservable“ funkcija koja omogućuje automatsko označavanje svih varijabli i metoda unutar store klase bez dodatnog ručnog označavanja. Slika 4.29 prikazuje sve implementirane store klase unutar aplikacije.



Slika 4.29: Prikaz svih store komponenti aplikacije

Uloga store klase je postavljanje vrijednosti dobivenih od backend dijela aplikacije na definirane „observable“ varijable kako bi ih se moglo koristiti dalje u aplikaciji. Moguće ih je koristiti na više mjesta pozivom postavljene vrijednosti iz store-a. Korištenje vrijednosti sa store-a omogućuje se da u istom trenutku na svakom dijelu aplikacije dobije jednaka trenutna vrijednost podataka. Na slici 4.30 prikazano je korištenje „makeAutoObservable“ funkcije gdje se u konstruktoru predaje parametar „this“ za automatsko označavanje varijabli i metoda u store klasi. Kao „observable“ varijable u ovom slučaju su varijable klase „aktivno“, „trajanje“, „start“, „rezultati“, „jeGlasao“. Metoda „ucitajStatus“ i druge metode koje se nalaze unutar klase su postavljene kao „action“ budući da mijenjaju stanje „observable“ varijabli. Ostale metode klase koje ne bi mijenjale vrijednosti varijabli, već samo koristile njihove vrijednosti bile bi označene sa „computed“.

```

src > stores > JS GlasanjeStore.js > GlasanjeStore > glasaj
1  import { makeAutoObservable } from 'mobx';
2  import { GlasanjeService } from '../api/services/GlasanjeService';
3
4  class GlasanjeStore {
5      aktivno = false;
6      trajanje = 0;
7      start = null;
8      rezultati = [];
9      jeGlasao = false;
10
11     constructor() {
12         makeAutoObservable(this);
13     }
14
15     async ucitajStatus(tockaId) {
16         const res = await GlasanjeService.getStatus(tockaId);
17         this.aktivno = res.data.aktivno;
18         this.trajanje = res.data.trajanje;
19         this.start = res.data.start;
20     }
21

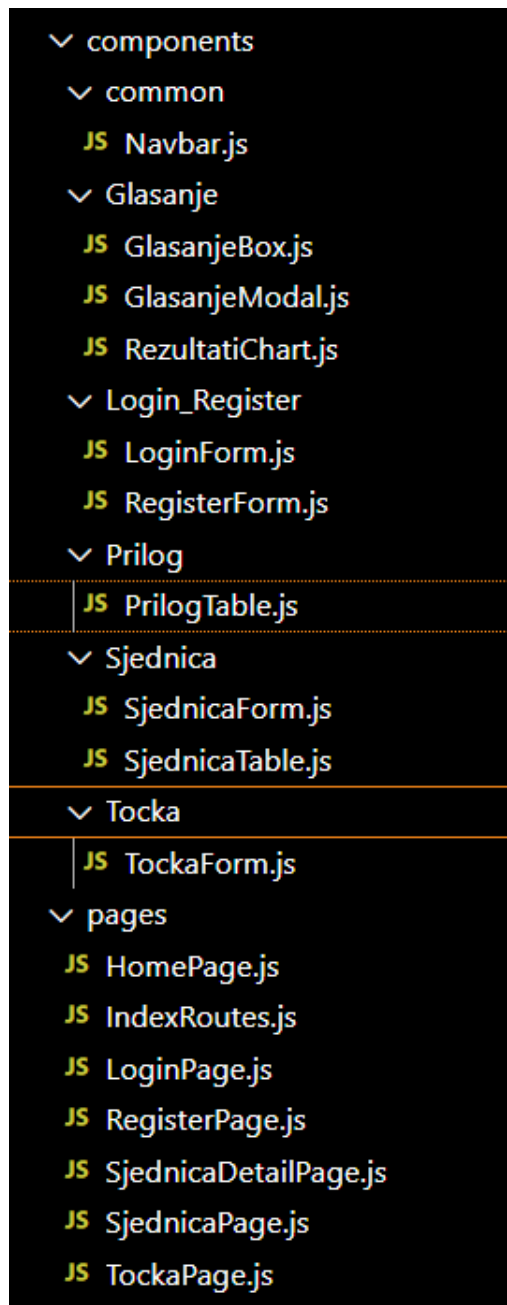
```

Slika 4.30: Prikaz store komponente za glasanje

Metoda „ucitajStatus“ za točku dnevnog reda korištenjem servisne klase dohvaća status s backend dijela aplikacije. Nakon što dobije odgovor, na „observable“ varijable postavlja dobivenu vrijednost za pripadajuće varijable. Na ovaj način omogućeno je korištenje istih vrijednosti bilo gdje unutar komponenti i stranica frontend aplikacije uvozom store klase. Također, korištenjem klasa bilo gdje u *kodu* moguće je pozivanje „action“ metoda za izmjenu ili ažuriranje varijabli.

4.3.3. Komponentne i konkretne stranice

Izrada korisničkog sučelja temelji se na izgradnji komponenti koje će se kasnije kombinirati s ostalim html sadržajem za implementaciju konačnog izgleda pojedinačnih stranica. Svaka komponenta odgovorna je samo za određeni dio koji prikazuje, dok se stranice onda organiziraju tako da predstavljaju logičke cjeline aplikacije, kao što je prikaz svih sjednica, detalja pojedine točke dnevnog reda ili same stranice za prijavu. Struktura projekta je organizirana tako da se komponente nalaze unutar posebne mape od konkretnih stranica vidljivo na slici 4.31.



Slika 4.31: Raspodjela strukture komponenti i stranica

Za svaku stranicu koja koristi komponente stvorena je mapa unutar koje se nalaze njezine pripadajuće komponente. Ukoliko se komponenta koristi na više različitih stranica smještena je unutar mape „common“. Jedna komponenta najčešće ima samo jednu funkcionalnost. Korištenjem manjih komponenti postiže se veća preglednost *koda* i smanjenje dupliciranog *koda*. Dobrom organizacijom komponenti olakšava se slaganje većih stranica ili većih komponenti, tako da se raspodjeli posao na više koraka koji zajedno čine cjelinu. „SjednicaPage“ konkretno koristi dvije komponente čija kombinacija rezultira konačnim izgledom stranice. Stranica se sastoji od html

komponenti koje čine naslov stranice i elemente za upravljanje filtriranja i odabira željenog broja podataka koji se prikazuju unutar sadržaja tablice, prikazano na slici 4.32.

```
57 <div className="sjednice-container">
58   <h2>Popis sjednica</h2>
59
60   <div className="sjednice-controls">
61     <input
62       type="text"
63       placeholder="Pretraži po nazivu ili lokaciji..."
64       value={search}
65       onChange={(e) => setSearch(e.target.value)}
66     />
67     <p></p>
68     <select
69       value={itemsPerPage}
70       onChange={(e) => {
71         setItemsPerPage(Number(e.target.value));
72         setCurrentPage(0);
73       }}
74     >
75       <option value={10}>Prikaži po stranici</option>
76       <option value={5}>5</option>
77       <option value={10}>10</option>
78       <option value={20}>20</option>
79     </select>
80     {isAdmin && (
81       <button className="add-button" onClick={() => setIsAddModalOpen(true)}>+ Dodaj sjednicu</button>
82     )}
83   </div>
84
```

Slika 4.32: Elementi za naslovni dio i filtriranje tablice na stranici sjednice

Nakon što se prikažu navedeni elementi prikazuje se komponenta zaslužna za prikaz svih sjednica unutar tablice, prikazano na slici 4.33. Kao parametri komponente proslijeđuju se trenutni sadržaj sjednica za prikaz, metode za određene akcije koje se nude u tablici i parametri potrebni za upravljanje sortiranjem na tablici.

```
84
85 <SjednicaTable
86   sjednice={currentItems}
87   onEdit={(sjednica) => {
88     setSelected(sjednica);
89     setIsEditModalOpen(true);
90   }}
91   onDelete={(sjednica) => {
92     if (window.confirm(`Želite li obrisati sjednicu "${sjednica.naziv}"?`)) {
93       sjednicaStore.delete(sjednica.id);
94     }
95   }}
96   onSort={toggleSort}
97   sortKey={sortKey}
98   sortAsc={sortAsc}
99 />
100
```

Slika 4.33: Korištenje tablica komponente za sjednicu

Nakon komponente za tablicu, prikazuju se elementi za upravljanje i prikaz stranica unutar tablice, prikazano na slici 4.34.

```
101
102     <div className="pagination-controls">
103       <button onClick={() => goToPage(currentPage - 1)} disabled={currentPage === 0}>
104         ← Prethodna
105       </button>
106       {Array.from({ length: pageCount }, (_, i) => (
107         <button
108           key={i}
109           onClick={() => goToPage(i)}
110           className={i === currentPage ? 'active' : ''}
111         >
112           {i + 1}
113         </button>
114       ))}
115       <button onClick={() => goToPage(currentPage + 1)} disabled={currentPage >= pageCount - 1}>
116         Sljedeća →
117       </button>
118     </div>
119
```

Slika 4.34: Elementi za upravljanje listanja stranica unutar tablice

Na kraju stranice koristi se komponenta za prikazivanje sjednica forme za uređivanje i dodavanje sadržaja. Korištenje komponenti prikazano je na slici 4.35.

```
120     {isAddModalOpen && (
121       <div className="modal-overlay">
122         <div className="modal-content">
123           <button className="close-modal" onClick={() => setIsAddModalOpen(false)}>✕</button>
124           <SjednicaForm onSuccess={() => setIsAddModalOpen(false)} />
125         </div>
126       </div>
127     )}
128
129     {isEditModalOpen && selected && (
130       <div className="modal-overlay">
131         <div className="modal-content">
132           <button className="close-modal" onClick={() => setIsEditModalOpen(false)}>✕</button>
133           <SjednicaForm sjednica={selected} onSuccess={() => setIsEditModalOpen(false)} />
134         </div>
135       </div>
136     )}
137   </div>
138 );
139 });
140
141 export default SjednicaPage;
142
```

Slika 4.35: Korištenje komponentne za formu sjednicu

Kao što je vidljivo na slici 4.35 komponenta za formu sjednica koristi se dva puta, prvi put je korištena za potrebe dodavanja nove sjednice u tablicu, dok je drugi puta korištena za promjenu

podataka odabrane sjednice iz tablice. Forme su korištene tako da se tek prilikom pritiska na gumb za željenu akciju prikazuje konkretna forma u obliku modalnog prozora. Prilikom odabira za ažuriranje postojeće sjednice, podaci se spremaju na vrijednost „selected“ i proslijeđuju formi za prikazivanje postojećih podataka unutar komponente. Postignuto je da se korisniku pokazuju trenutne vrijednosti sjednice i mogućnost izmjenjivanja istih. Prilikom dodavanja nove sjednice prikazuje se prazna forma i korisnik unosi nove podatke za pohranu u bazu podataka.

```

12  const SjednicaDetailPage = observer(() => {
57
58
59    return (
60      <div className="sjednice-container">
61        <div className="sjednice-header" style={{ display: 'grid', justifyContent: 'space-between', alignItems: 'center' }}>
62          <h2>{sjednicaStore.sjednica.naziv}</h2>
63          {isAdmin && (
64            <button className="add-button" style={{ display: "inline - flex", alignItems: 'center', justifyContent: 'center' }}
65              onClick={() => setIsEditModalOpen(true)}>✎ Uredi sjednicu</button>
66          )}
67        </div>
68
69        {isEditModalOpen && (
70          <div className="modal-overlay">
71            <div className="modal-content">
72              <button className="close-modal" onClick={() => setIsEditModalOpen(false)}>✕</button>
73              <SjednicaForm sjednica={sjednicaStore.sjednica} onSuccess={() => setIsEditModalOpen(false)} />
74            </div>
75          </div>
76        )}
77
78        {
79          (selectedTocka || isAddTockaModalOpen) && (
80            <div className="modal-overlay">
81              <div className="modal-content">
82                <button className="close-modal" onClick={() => { setSelectedTocka(null); setIsAddTockaModalOpen(false); }}>✕</button>
83                <TockaForm
84                  tocka={selectedTocka}
85                  sjednicaId={id}
86                  onSuccess={async () => {
87                    await tockaStore.fetchBySjednicaId(id);
88                    setSelectedTocka(null);
89                    setIsAddTockaModalOpen(false);
90                  }}
91                />
92              </div>
93            </div>
94          )
95        }
96      </div>
97    );
98  });

```

Slika 4.36: Korištenje sjednica i točka forme na drugim stranicama

Slika 4.36 prikazuje korištenje komponenti za formu sjednice i točke dnevnog reda unutar različitih stranica aplikacije. Prilikom odabira konkretne sjednice iz tablice otvara se nova stranica koja prikazuje detalje te sjednice, što uključuje podatke o sjednici i točkama dnevnog reda. U tom slučaju je potrebno na stranici omogućiti izmjenjivanje jednog i drugog dijela. Budući da se preko store klasa pristupa podacima odabrane sjednice i točke dnevnog reda, omogućeno je ponovno korištenje komponente za prikazivanje forme sjednica i točki dnevnoga reda. Kombinacija korištenja sličnih komponenti i skupa html elemenata uz njih rezultira kompletnom verzijom frontend dijela web aplikacije. Uz komponente za stvaranje tablica i formi za razne tipove podataka, razlikuje se komponenta zaslužna za prikaz rezultata u obliku grafikona. Za potrebe implementacije grafikona korištena je „recharts“ biblioteka koja za predane glasove prebrojava

svaki tip i rezultate prikazuje grafički. Implementacija komponente za grafički prikaz rezultata prikazana je na slici 4.37.

```
src > components > Glasanje > JS RezultatiChartjs > ...
1  import React from 'react';
2  import { BarChart, Bar, XAxis, YAxis, Tooltip, CartesianGrid, ResponsiveContainer } from 'recharts';
3  import { observer } from 'mobx-react-lite';
4  import { glasanjeStore } from '../stores/GlasanjeStore';
5
6  const RezultatiChart = observer(() => {
7    const countGlasova = (tip) =>
8      glasanjeStore.rezultati.filter(g => g.glas === tip).length;
9
10   const data = [
11     { ime: 'ZA', glasova: countGlasova('ZA') },
12     { ime: 'PROTIV', glasova: countGlasova('PROTIV') },
13     { ime: 'SUZDRŽAN', glasova: countGlasova('SUZDRŽAN') }
14   ];
15
16   return (
17     <div style={{ width: '100%', height: 300 }}>
18       <h3>Rezultati glasanja</h3>
19       <ResponsiveContainer>
20         <BarChart data={data}>
21           <CartesianGrid strokeDasharray="3 3" />
22           <XAxis dataKey="ime" />
23           <YAxis allowDecimals={false} />
24           <Tooltip />
25           <Bar dataKey="glasova" fill="#0d6efd" />
26         </BarChart>
27       </ResponsiveContainer>
28     </div>
29   );
30 });
31
32 export default RezultatiChart;
33
```

Slika 4.37: Implementacija grafičkog prikaza za rezultate glasanja


```

    {glasanjeStore.aktivno && preostalo !== null && (
      <div className="glasanje-timer-box">
        <p style = {{marginTop : '2%'}}className="glasanje-timer-text">Preostalo vrijeme za glasanje: <strong>{preostalo}s</strong></p>
      </div>
    )}

    {glasanjeStore.aktivno && !vrijemeZavrшено() && (
      glasanjeStore.jeGlasao ? <p>Glasanje evidentirano za ovaj račun.</p> :
      <button className="add-button" onClick={() => setShowModal(true)}>Glasaj</button>
    )}

    {glasanjeStore.start &&
      Date.now() > new Date(glasanjeStore.start).getTime() + glasanjeStore.trajanje * 1000 && (
        <div>
          <h3>Rezultati:</h3>
          <ul>
            <li>ZA: {glasanjeStore.rezultati.filter(g => g.glas === "ZA").length}</li>
            <li>PROTIV: {glasanjeStore.rezultati.filter(g => g.glas === "PROTIV").length}</li>
            <li>SUZDRŽAN: {glasanjeStore.rezultati.filter(g => g.glas === "SUZDRŽAN").length}</li>
          </ul>
          <RezultatiChart />
        </div>
      )}

    {showModal && <GlasanjeModal tokenId={tokenId} onClose={() => setShowModal(false)} />}
  </div>
);
});
export default GlasanjeBox;

```

Slika 4.38: Komponenta za prikaz glasanja na stranici točki dnevnoga reda

Komponenta „GlasanjeBox“ sastoji se od potrebnih html elemenata koje ulozi administrator omogućuje pokretanje glasanja. Nakon što je glasanje pokrenuto započinje odbrojavanje unesenog vremena. Pritiskom na gumb „glasaj“, korisniku se otvara modalni prozorčić, odnosno komponenta „GlasanjeModal“ putem koje može glasati. Nakon isteka vremena korisniku se onemogućuje glasanje te se na zaslonu prikazuju rezultati glasanja. Komponenta „GlasanjeBox“ prikazana je na slici 4.38. Komponenta „PrilogTable“ također se nalazi na stranici za prikazivanje točki dnevnoga reda kao zadnja komponenta stranice. Prilozi su prikazani u obliku tablice sa svim unesenim prilogima. Administrator upravlja postavljanjem i brisanjem dokumenata unutar tablice. On unosi dokumente u tablicu koji se mogu kasnije preuzimati. Primjer provjere za unesene dokumente i prosljeđivanje dokumenta na pripadajuću store klasu prikazan je na slici 4.39.

```
const handleUpload = async (file) => {
  if (!file) return alert('Odaberi datoteku');

  const allowedExtensions = ['pdf', 'doc', 'docx', 'xlsx', 'xls', 'txt', 'csv', 'json', 'jpg', 'jpeg', 'png'];
  const fileExtension = file.name.split('.').pop().toLowerCase();

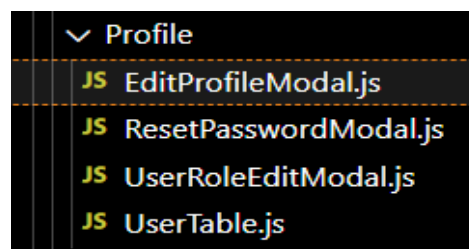
  if (!allowedExtensions.includes(fileExtension)) {
    |   return alert('Nepodržani format datoteke.');
```

```
  }

  const formData = new FormData();
  formData.append('file', file);
  await prilogStore.upload(tokenId, formData);
  setFileName('');
```

Slika 4.39: Kod za upravljanje unošenja priloga unutar PrilogTable komponente

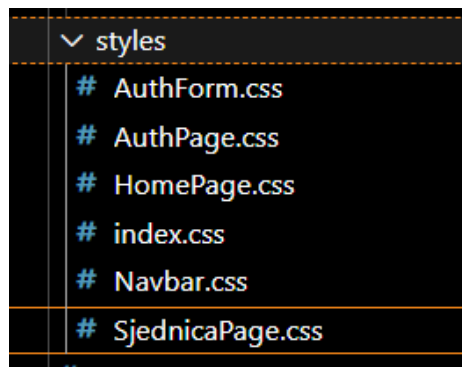
Posebne komponente stvaraju se za potrebe prikazivanja i uređivanja korisnikovih podataka na profilu. Popis korištenih komponenti prikazan je na slici 4.40. Svaka od komponenti ima svoju ulogu za prikaz na stranici. Komponenta „EditProfileModal“ koristi se za potrebe modala kako bi omogućili promjenu korisnikovih podataka, dok se posebni modal „ResetPasswordModal“ koristi za promjenu korisnikove lozinke. „UserTable“ komponenta koristi se za prikaz svih korisničkih profila unutar aplikacije i vidljiva je samo administrator i predlagatelj ulozi. Njima je omogućeno mijenjanje korisničkih uloga kroz posebni modal „UserRoleEditModal“.



Slika 4.40: Prikaz komponenti korištenih za korisnikov profil

4.3.4. Dizajn stranice

Konačni izgled korisničkog sučelja definiran je pomoću vlastitih CSS datoteka organiziranih prema funkcionalnosti pojedinih stranica i komponenti. Popis stvorenih CSS datoteka prikazan je na slici 4.41.



Slika 4.41: Struktura CSS datoteka

Korištenjem sistemskog „sans-serif“ fonta unutar cijele aplikacije izbjegnuto je dodatno učitavanje vanjskih web fontova što rezultira bržim prikazom između stranica. Dizajn cijele stranice baziran je na nijansama plave boje, koje dominiraju sučeljem i daju mu profesionalan i konzistentan izgled. Primarne nijanse plave boje bile bi #074281 i #0056b3, koje su korištene kao pozadinske boje elemenata te njihova kombinacija za isticanje obruba kartica i drugih elemenata, poput gumba i navigacijske trake. Veoma svijetlije nijansa plave boje, poput #f1f9ff i #f0f8ff, korištene su kao pozadinske boje kartica i unutrašnjosti formi. Boje teksta definirane su nijansama crne, #333 i #000, kako bi osigurale visoku čitljivost na svijetlim pozadinama. Za prikaz stanja pogrešaka i negativnih akcija, kao što su odbijanje zahtjeva ili prikaza validacijskih poruka, primijenjene su nijanse crvene #dc3545 i #bb2d3b. Time se jasno vizualno razlikuju od ostalih elemenata. Moderni izgled sučelja dodatno je postignut korištenjem umjerenih zaobljenja rubova elemenata (4px, 6px i 8px) te blažih sjena koje omogućuju osjećaj dubine, a ne ometaju preglednost sadržaja. Flexbox i CSS-grid korišteni su za centriranje formulara i raspoređivanje kartica [8].

```
.sjednica-form button:hover {
  background-color: #0056b3;
}

.s.btn-table.edit.active {
  background-color: #032e6d !important;
  color: white;
  font-weight: bold;
  border: 2px solid #074281;
  box-shadow: 0 0 6px rgba(0, 0, 0, 0.2);
}

.btn-table.delete {
  background-color: #dc3545;
}

.btn-table.delete:hover {
  background-color: #bb2d3b;
}
```

Slika 4.42: Korištene boje unutar aplikacije

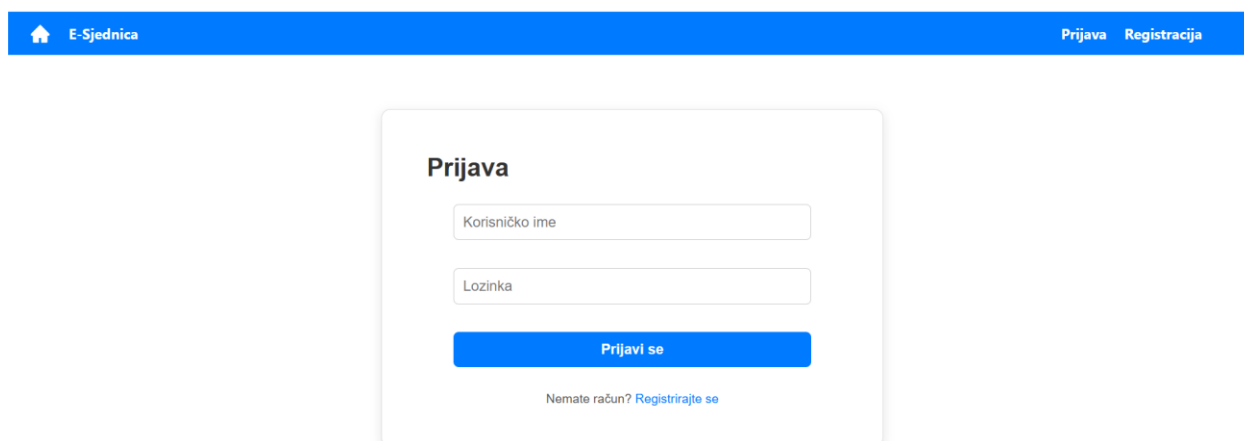
5. PRIKAZ RADA APLIKACIJE

Komponenta navigacijske trake uvijek je smještena na vrhu stranice s tamnijom pozadinom, logotipom i bojom teksta bijele boje unutar trake. Početna stranica web aplikacije prikazana je na slici 5.1.



Slika 5.1: Početna stranice web aplikacije

Korisnik se s početne stranice, za daljnje korištenje klikom na navigacijsku traku, prijavljuje ili registrira na stranicu. Prikaz stranice za prijavu prikazana je na slici 5.2.



Slika 5.2: Stranica za prijavu korisnika

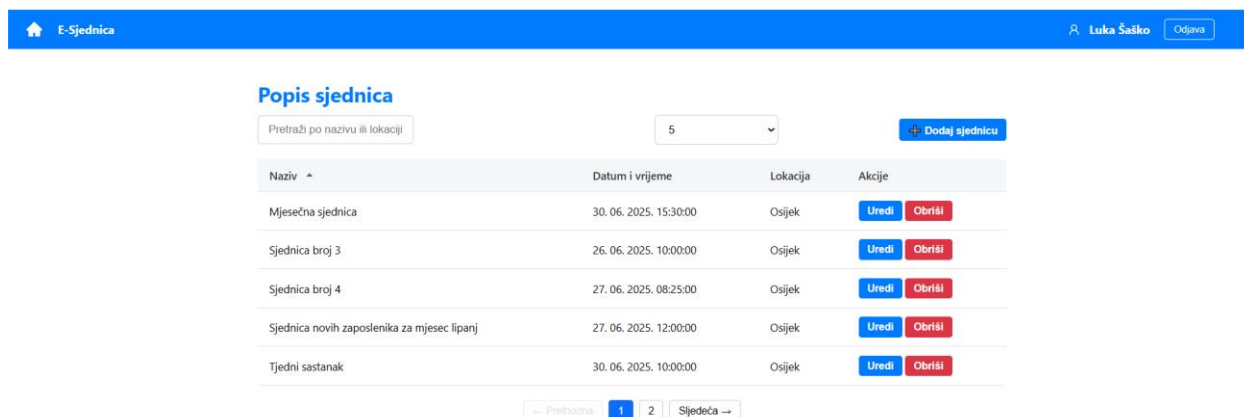
Ukoliko korisnik nema postojeći račun, pritiskom na poveznicu „Registrirajte se“ na dnu forme, korisnik se odvodi na formu za registraciju. Prikaz registracijske forme prikazan je na slici 5.3.

Slika 5.3: Registracijska forma

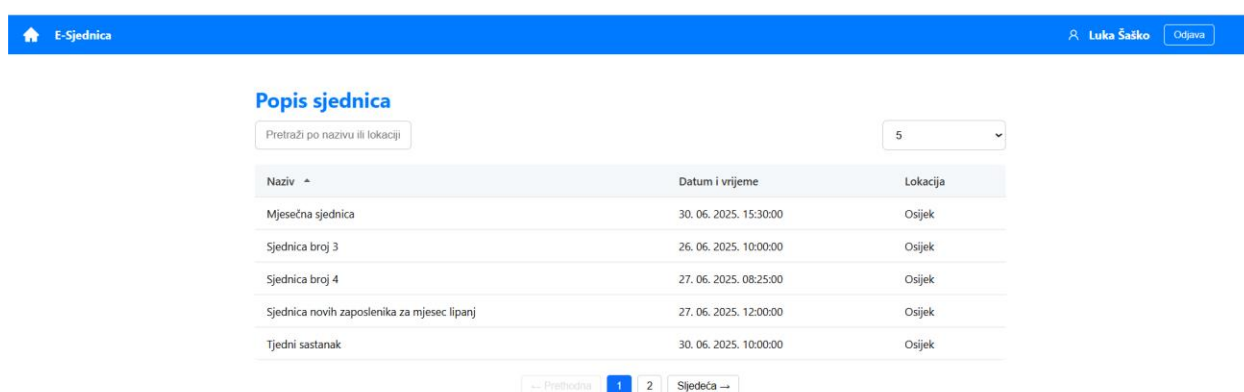
Korisnik prijelaz između stranica za prijavu i registraciju postiže pritiskom unutar navigacijske trake ili poveznice na rubu forme. Ukoliko korisnik koji nije prijavljen unutar navigacijske trake pritisne gumb „E-sjednica“ prikazuje se samo pred pregled tablice korištene za prikaz sjednica bez podataka, prikazano na slici 5.4.

Slika 5.4: Prikaz stranice sjednica za ne prijavljene korisnike

Nakon uspješne prijave ili registracije korisnika se automatski prebacuje na stranicu sa unesenim sjednicama. Korisnik ima uvid u sve sjednice koje su trenutno aktivne, a ovisno o njegovoj ulozi ograničene su mu akcije. Razlike između administratorske i predlagateljske uloge s ostalim ulogama prikazane su na slikama 5.5. i 5.6. Nakon prijave korisnikovo ime prikazano je u desnom rubu navigacijske trake kao i gumb za odjavu.

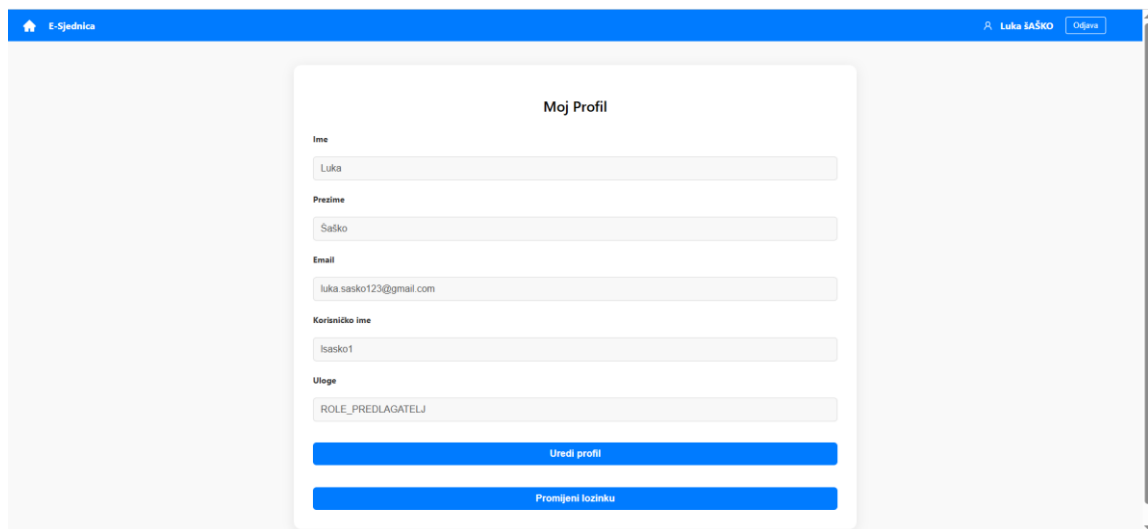


Slika 5.5: Prikaz stranice sjednice za administratora



Slika 5.6: Prikaz stranice sjednice za ostale uloge

Pritiskom na korisničko ime otvara se stranica s korisnikovim profilom. Korisniku je omogućeno uređivanje osobnih podataka kao i ažuriranje trenutne lozinke, prikazano na slici 5.7. Pritiskom na gumb „Uredi profil“ korisniku se otvara modalnih prozor čiju formu može uređivati te spremi promjene, prikazano na slici 5.8.



Moj Profil

Ime
Luka

Prezime
Šaško

Email
luka.sasko123@gmail.com

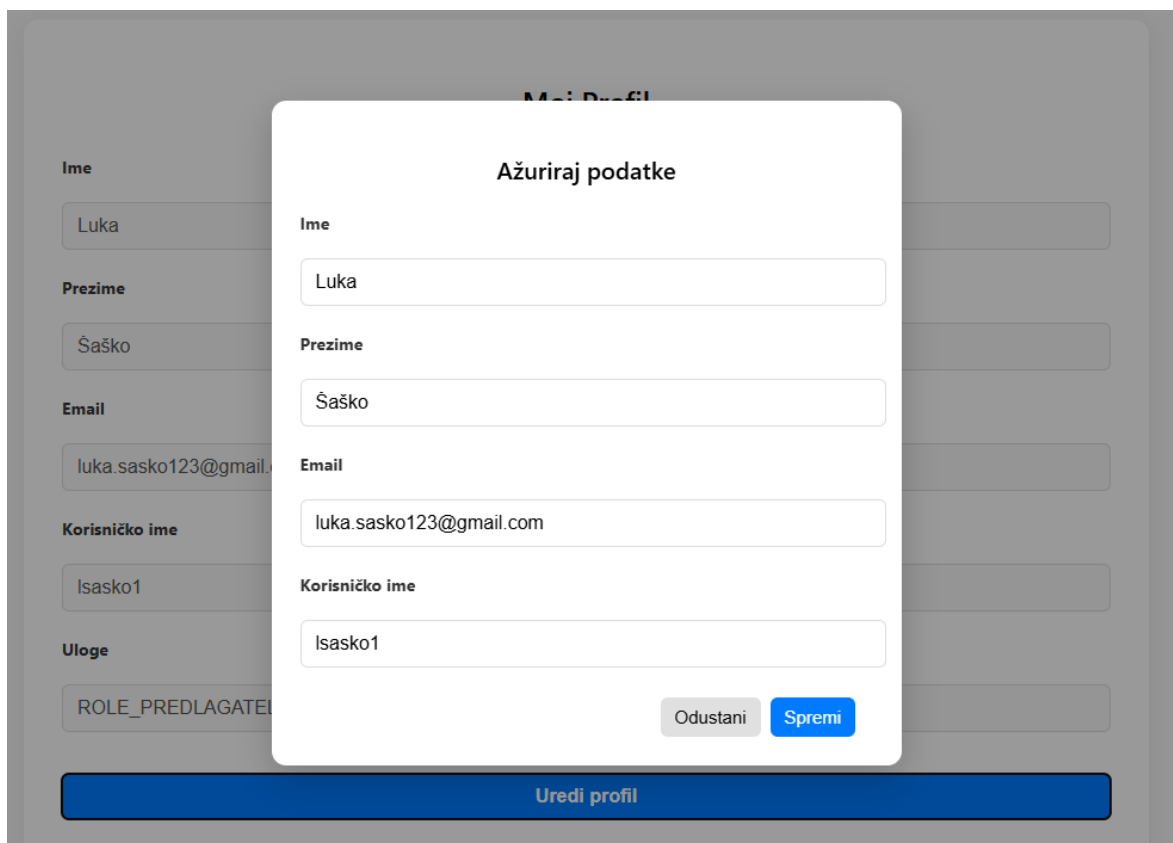
Korisničko ime
lsasko1

Uloge
ROLE_PREDLAGATELJ

Uredi profil

Promijeni lozinku

Slika 5.7: Prikaz stranice za korisnikov profil



Ažuriraj podatke

Ime
Luka

Prezime
Šaško

Email
luka.sasko123@gmail.com

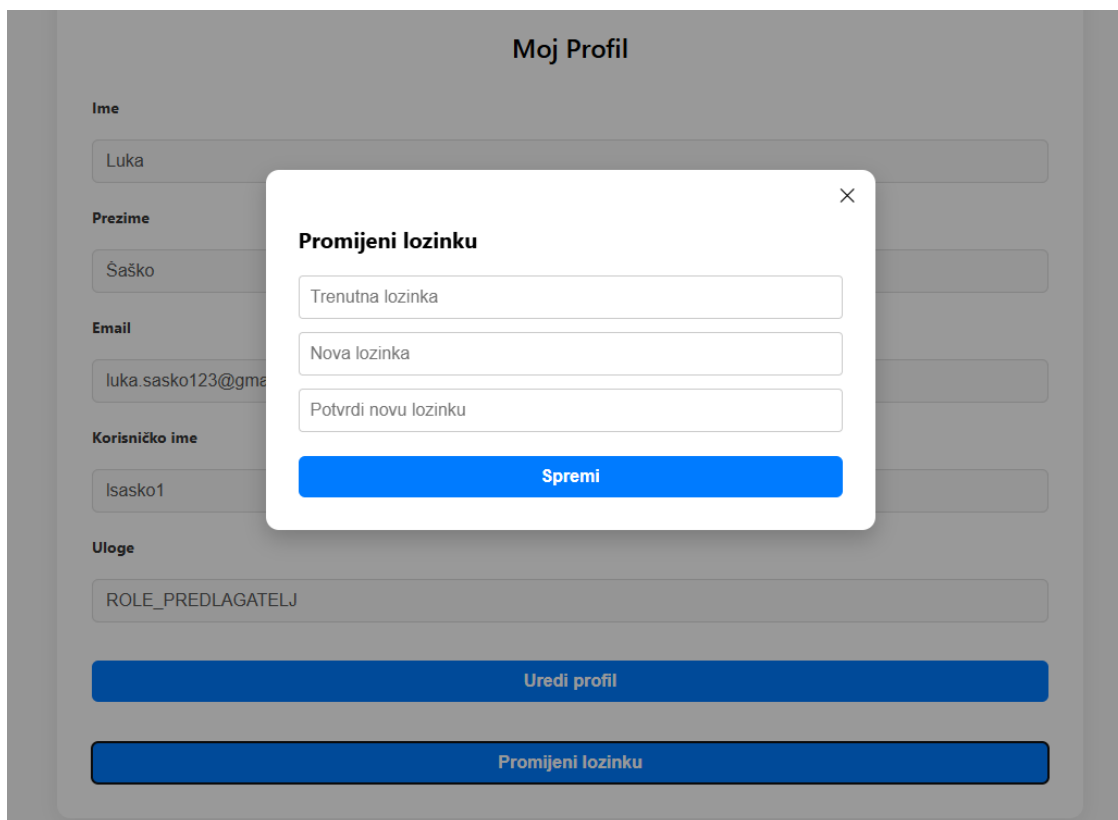
Korisničko ime
lsasko1

Odustani Spremi

Uredi profil

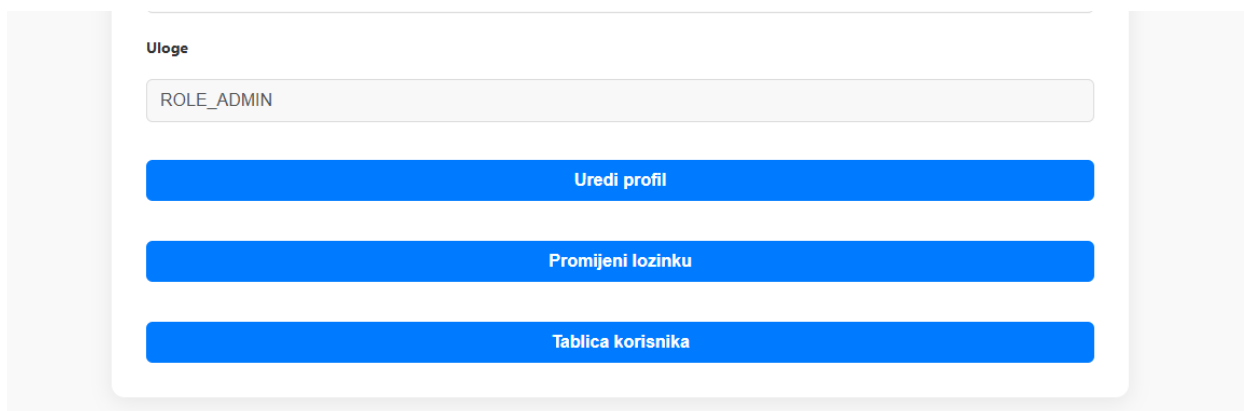
Slika 5.8: Prikaz modalnog prozora za uređivanje podataka profila

Za promjenu korisnikove lozinke potrebno je unijeti trenutnu aktivnu lozinku koja se koristi zajedno s novom željenom lozinkom te ponovnim upisom iste te lozinke za potvrdu. Prikazano na slici 5.9.

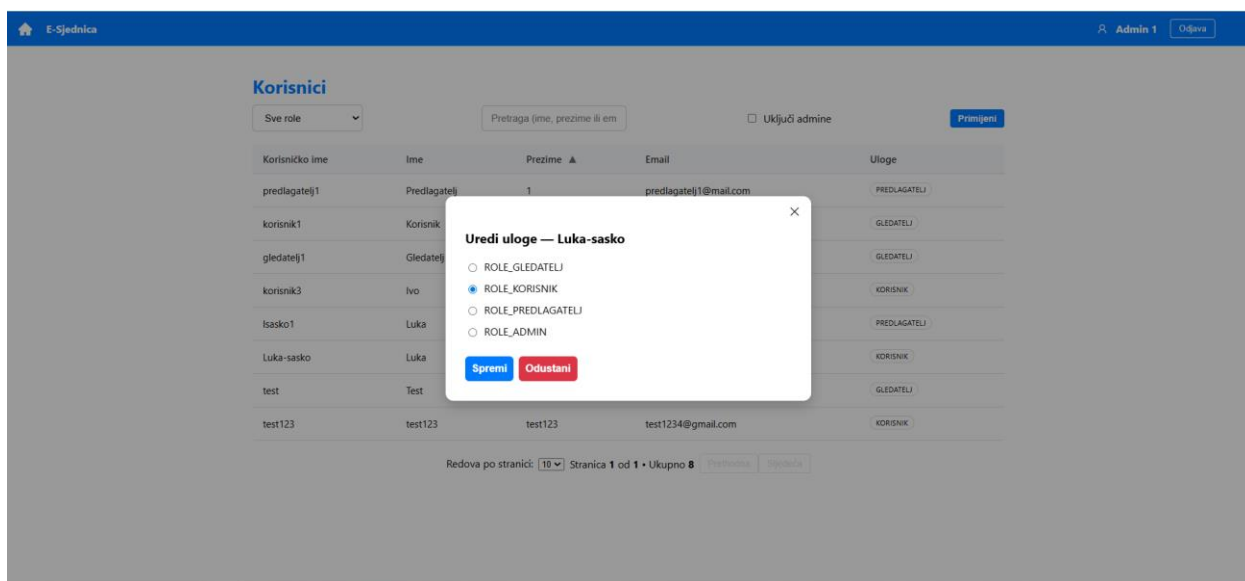


Slika 5.9: Prikaz modala za promjenu korisnikove lozinke

Ukoliko je prijavljeni korisnik administrator ispod gumba „Promijeni lozinku“ nalazi se dodatni gumb „Tablica korisnika“ koja prikazuje dodatnu tablicu trenutno registriranih korisnika. Administratoru je omogućena izmjena uloga svih korisnika aplikacije. Pritiskom na određenog korisnika otvara se modalni prozor s ponuđenim ulogama koje može dodijeliti odabranom korisniku. Prikazano na slikama 5.10 i 5.11.



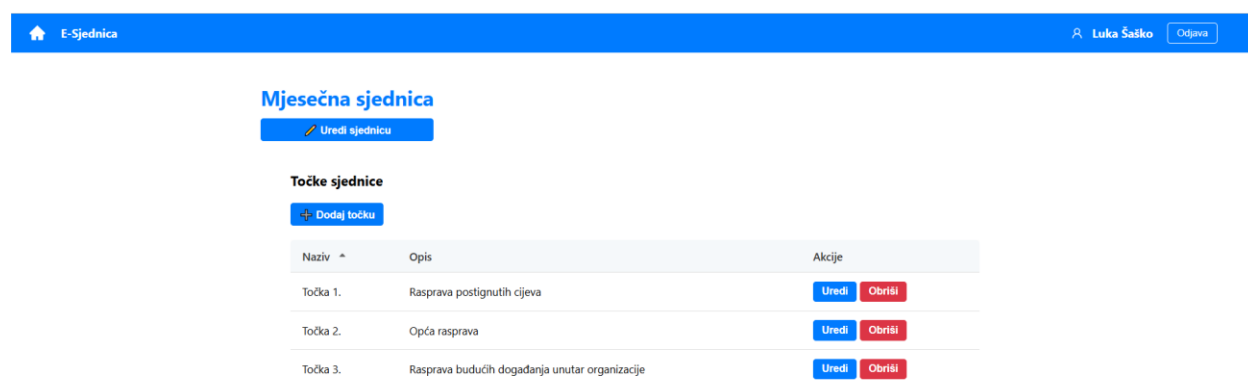
Slika 5.10: Prikaz stranica za ulogu administrator



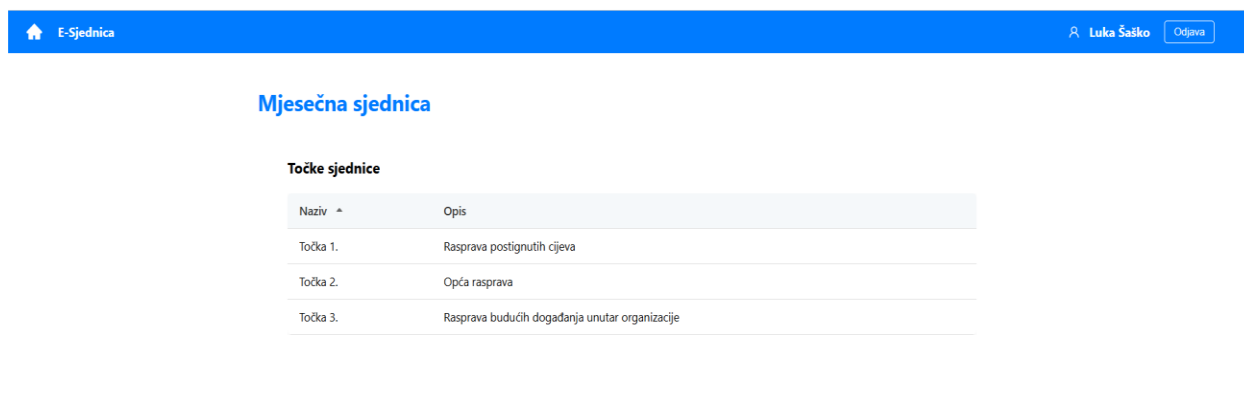
Slika 5.11: Prikaz stranice za prikaz tablice korisnika te modala za uređivanje uloga

Kao što je vidljivo na slici 5.11 stranica za prikaz tablice korisnika omogućuje pregled registriranih korisnika i pripadajućih podataka. Administratoru je omogućeno pretraživanje korisnika po imenu, prezimenu ili email adresi te je također omogućeno filtriranje podataka po korisničkim ulogama kao i opcija za prikaz ostalih administratorskih uloga. Pritiskom na korisnika otvara se modalni prozor te je omogućena izmjena uloge korisnika. Podaci se automatski ažuriraju kako se određeni filteri ili uneseni tekst za pretraživanje mijenja.

Ukoliko korisnik pritisne neku od sjednica, korisnika se prebacuje na stranicu za prikaz točki dnevnoga reda vezanih za tu sjednicu. Korisnici imaju uvid u sve točke dnevnoga reda, a ovisno o ulozi ograničene su im akcije na toj stranici. Prikaz njihovih razlika vidljiv je na slikama 5.12 i 5.13.

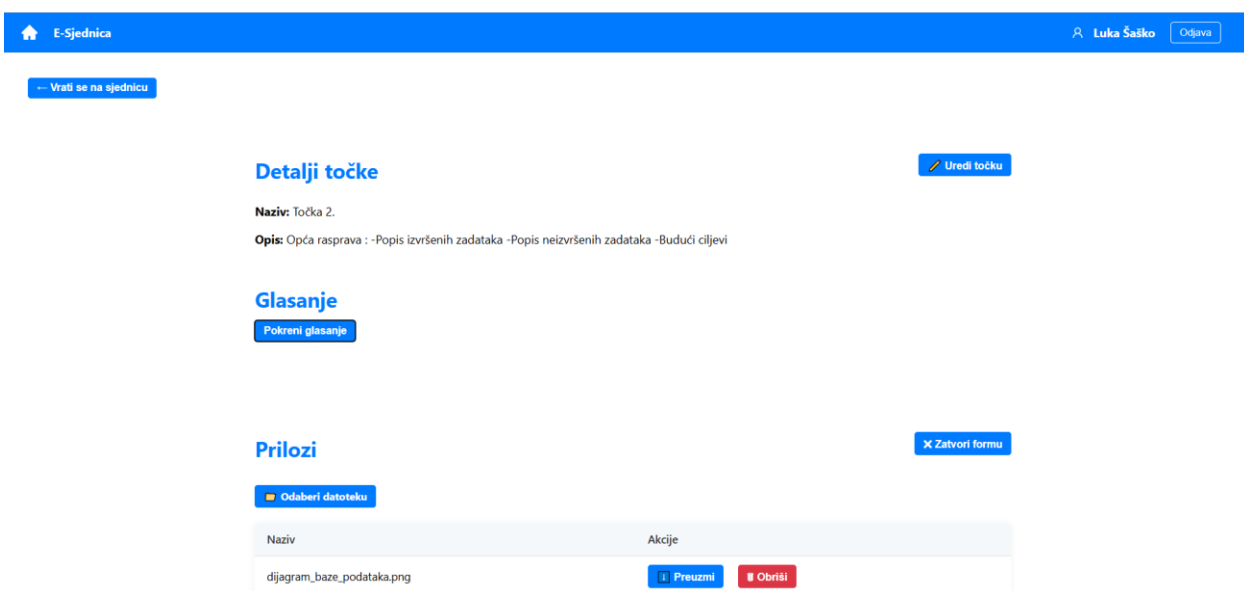


Slika 5.12: Administrator prikaz stranice za odabranu sjednicu

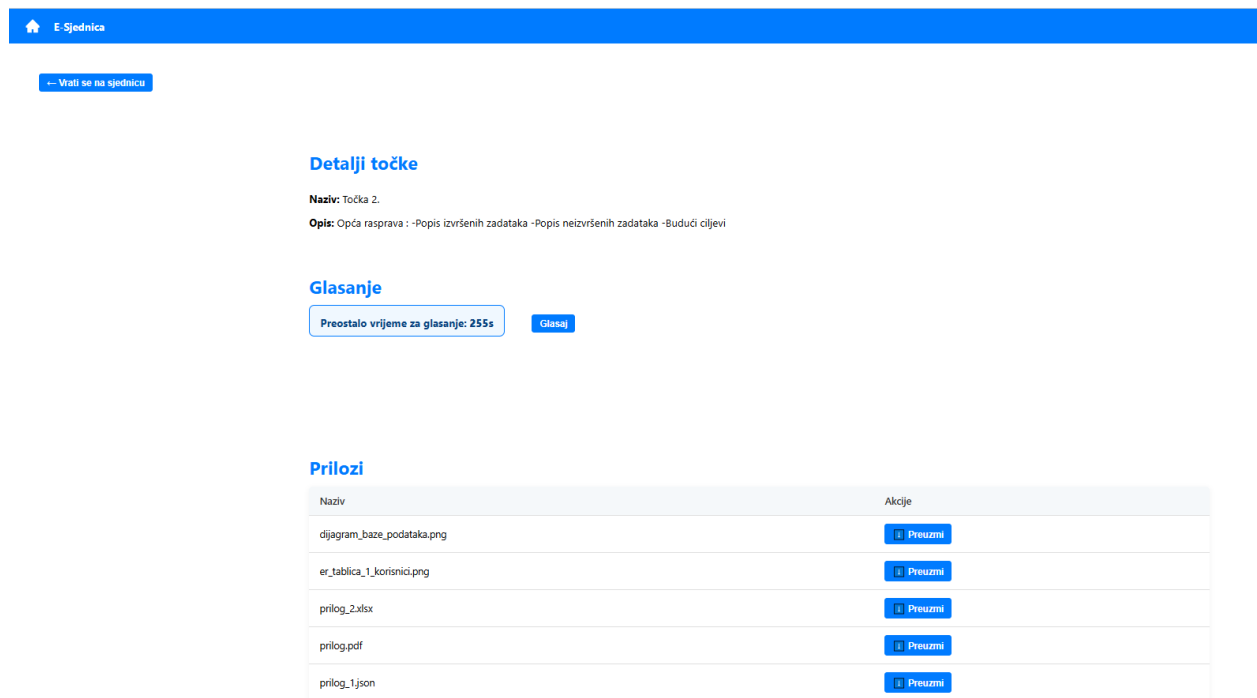


Slika 5.13: Prikaz stranice za odabranu sjednicu za ostale uloge

Glavna razlika u prikazu je što administratori i predlagatelji imaju mogućnost izmjene za sjednicu na kojoj se nalaze te mogućnosti dodavanja, uređivanje i brisanja točki dnevnoga reda, dok ostali mogu samo odabrati točku dnevnoga reda. Pritiskom na točku dnevnoga reda unutar tablice korisnike se prebacuje na stranicu za prikaz odabrane točke, postojećih priloga koji su dodani za tu točku te rezultati. Prikaz odabrane točke dnevnoga reda prikazan je na slikama 5.14 i 5.15 za različite uloge. Glavne razlike u prikazu su te da administrator i predlagatelj imaju mogućnost uređivanja kompletnih podataka na stranici kao i brisanja priloga. Također, oni moraju pokrenuti glasanje kako bi omogućili ostalim korisnicima da evidentiraju glas. Gledatelj ulozi nije vidljiva sekcija s glasanjem, već samo konačni rezultati glasanja. Prilikom pokretanja glasanja unosi se vremenski period trajanja u sekundama. Pokrenuto glasanje prikazano je na slici 5.15.

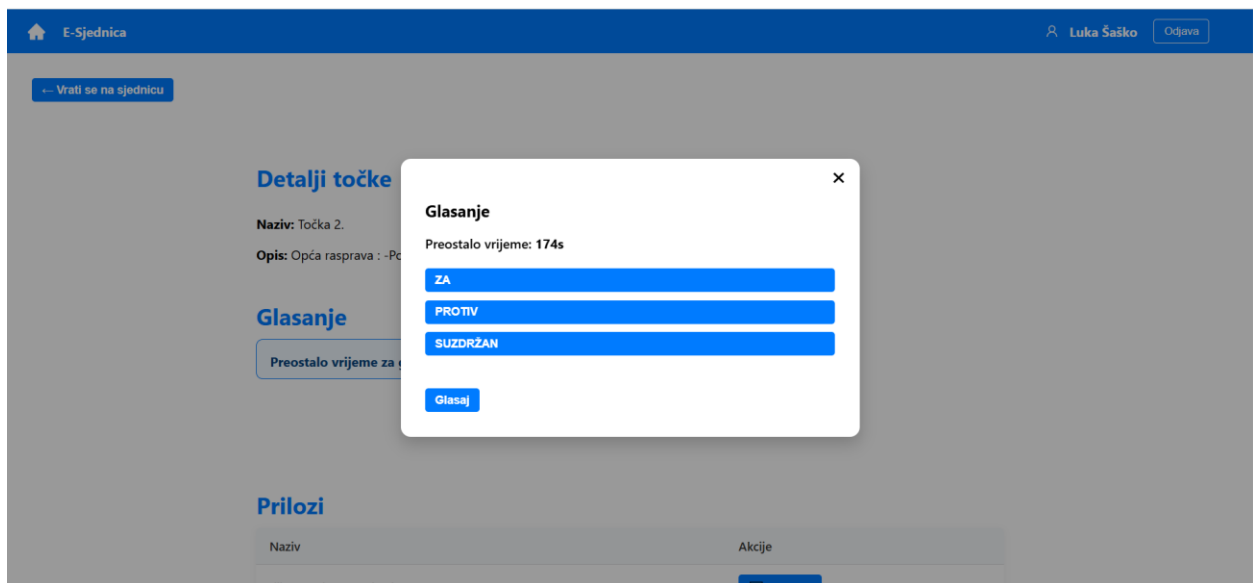


Slika 5.14: Prikaz stranice odabrane točke za administratora i predlagatelja

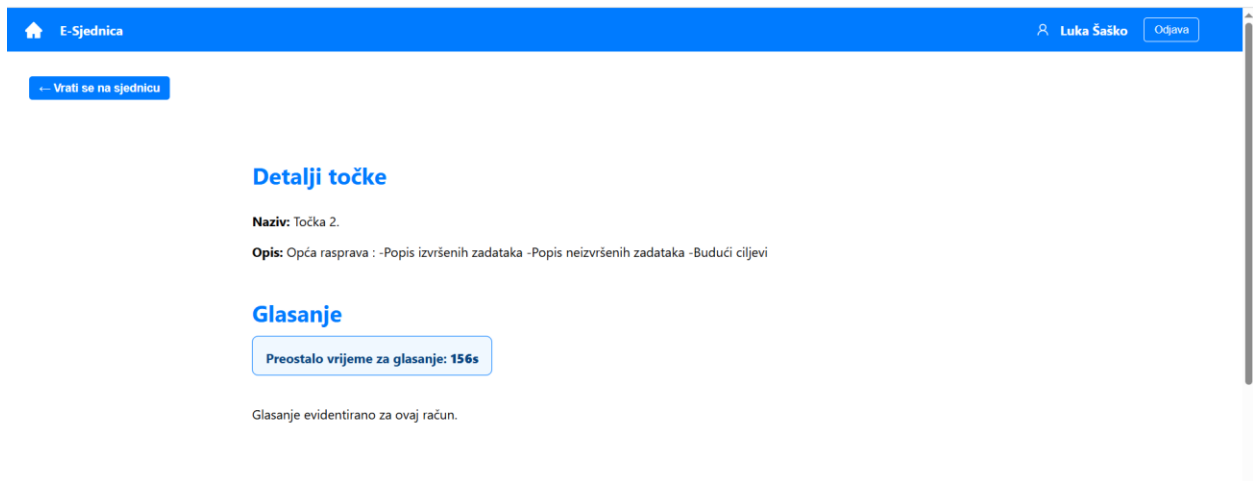


Slika 5.15: Prikaz stranice za odabranu točku za korisničku ulogu

Svim korisničkim ulogama omogućeno je preuzimanje priloga s točke. Prilozi mogu biti tekstualna datoteka, excel tablica, word dokument, json datoteka, csv datoteka, pdf datoteke ili slike raznih formata. Nakon što korisnik pritisne na gumb „Glasaj“ otvara mu se modalni prozor s ponuđenim opcijama koje može odabrati, prikazano na slici 5.16. Nakon što korisnik jednom glasa, odnosno potvrdi svoj glas pritiskom na gumb „Glasaj“, on nema više mogućnost izmjene svog glasa kao niti mogućnost ponovnoga glasanja. Korisnik čeka završetak glasanja, nakon čega se svim korisnicima prikazuju rezultati glasanja, prikazano na slikama 5.17 i 5.18.



Slika 5.16: Prikaz modala za glasanje



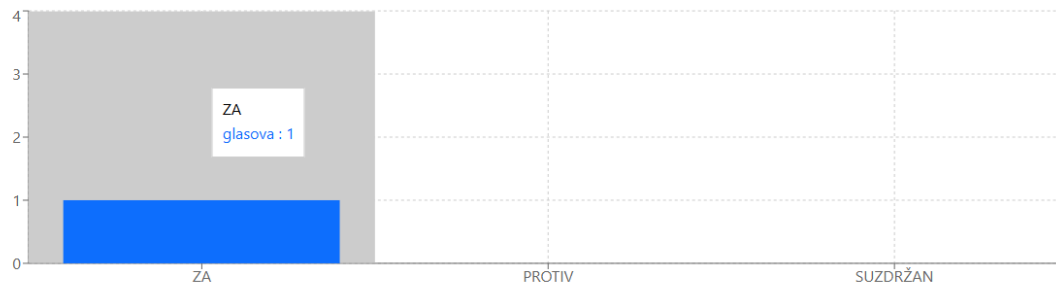
Slika 5.17: Prikaz stranice nakon glasanja

Glasanje

Rezultati:

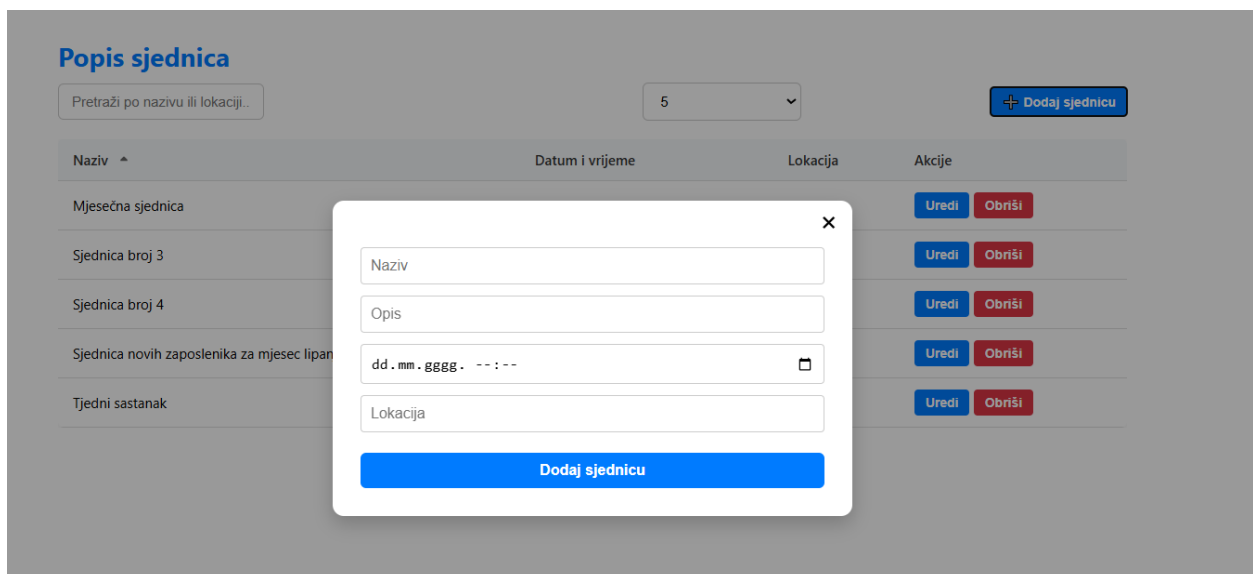
- ZA: 1
- PROTIV: 0
- SUZDRŽAN: 0

Rezultati glasanja

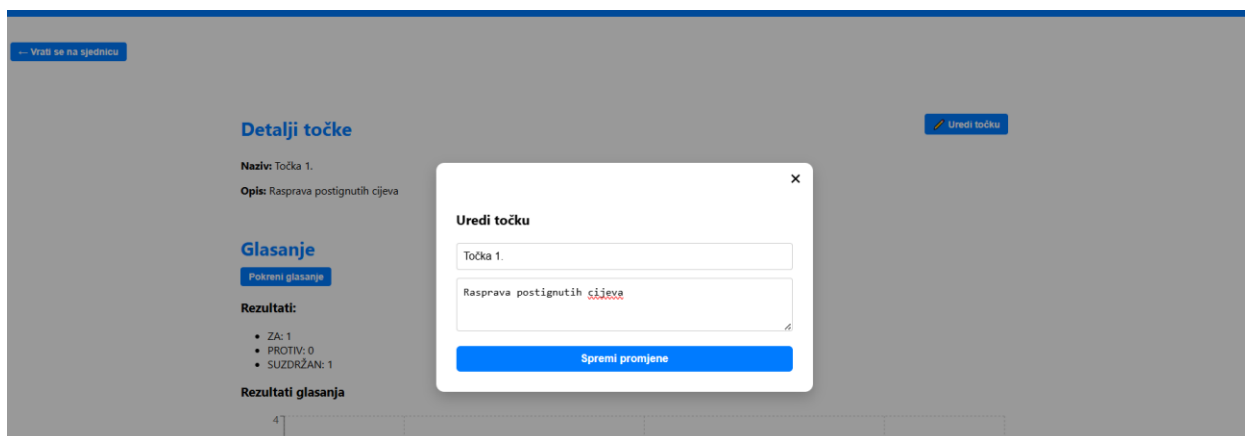


Slika 5.18: Prikaz rezultata glasanja

Administratoru se prilikom dodavanja ili izmjenjivanja podataka otvaraju modalni prozori s odgovarajućom formom. Ukoliko se dodaje novi element u tablicu, forma je prazna, odnosno prikazuje se bez podataka, kao što je vidljivo na slici 5.19. Ako se mijenjaju postojeći elementi iz tablice forma se automatski popunjava s trenutnim podacima, prikazano na slici 5.20.

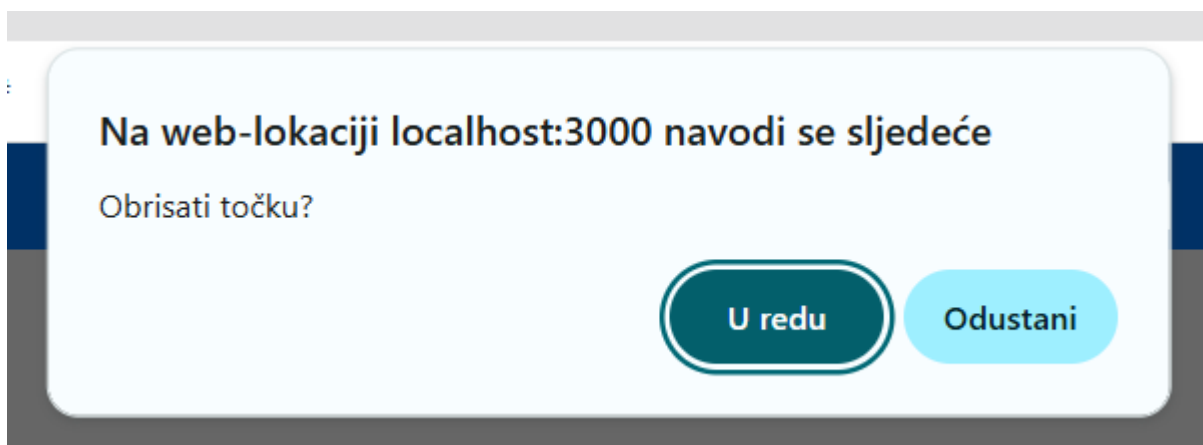


Slika 5.19: Dodavanje nove sjednice iz tablice



Slika 5.20: Izmjena postojeće točke dnevnoga reda

Prilikom brisanja podataka iz tablice, nakon pritiska gumba „Obriši“ potrebno je dodatno potvrditi tu akciju pritiskom gumba „U redu“ unutar prozora prikazanog na slici 5.21.



Slika 5.21: Prikaz potvrdnog prozora za brisanje

6. ZAKLJUČAK

Aplikacija je razvijena koristeći suvremene tehnologije na backend i frontend dijelu. Poseban naglasak bio je na samu funkcionalnost, sigurnost i prikaz korisničkog sučelja. Backend dio aplikacije implementiran je korištenjem programskog jezika Java te Spring Boot okvira. Backend osigurava autentifikaciju korisnika koristeći JWT mehanizam, ovisno o prijavljenoj ulozi kontrolira pristup resursima te komunicira s relacijskom bazom podataka PostgreSQL putem Spring Dana JPA. Za obavještanje korisnika u stvarnom vremenu o pokretanju glasanja korišten je WebSocket protokol. Dok je frontend dio web aplikacije razvijen na bibliotekama ReactJS i MobX za upravljanje stanjima unutar aplikacije. Za potrebe komuniciranja s backend dijelom aplikacije korištena je biblioteka Axios. Web aplikacije raspodijeljena je na nekoliko ključnih komponenti koje se prikazuju na konkretnim stranicama. Dizajn samog sučelja temelji se na jednostavnim i preglednim stranicama. Stiliziranje aplikacije provedeno je modularnim CSS datotekama s naglaskom na preglednost, kontrast i pristupačnost korisnicima. Implementirani sustav omogućuje administratorima da stvore sjednice, definiraju specifične točke dnevnoga reda unutar kojih mogu započeti glasanje i dodavati priloge. Korisnicima je omogućeno glasanje za svaku točku dnevnoga reda, uvid u rezultate i mogućnost preuzimanja priloženih datoteka. Sam sustav je dizajniran tako da se maksimalno smanji mogućnost pogrešaka, kao što je višestruko glasanje ili glasanje nakon isteka vremenskog okvira.

LITERATURA

- [1] Pivotal Software, Inc., Spring Boot Reference Documentation [online], Pivotal Software, 2024, dostupno na: <https://docs.spring.io/spring-boot/docs/current/reference/html/> [Pristupljeno: 25.4.2025.]
- [2] Pivotal Software, Inc., Spring Security Documentation [online], Pivotal Software, 2024, dostupno na: <https://docs.spring.io/spring-security/reference/> [Pristupljeno: 5.4.2025.]
- [3] Meta Platforms, Inc., React Documentation [online], Meta Open Source, 2024, dostupno na: <https://react.dev/> [Pristupljeno: 15.6.2025.]
- [4] PostgreSQL Global Development Group, PostgreSQL Documentation [online], PostgreSQL Global Development Group, 2024, dostupno na: <https://www.postgresql.org/docs/> [Pristupljeno: 2.4.2025.]
- [5] Auth0 Inc., *Introduction to JSON Web Tokens* [online], Auth0, 2023, dostupno na: <https://jwt.io/introduction> [Pristupljeno: 18.4.2025.]
- [6] MobX Team, MobX: Simple, scalable state management [online], MobX, 2024, dostupno na: <https://mobx.js.org/> [Pristupljeno: 15.6.2025.]
- [7] Axios Team, *Axios Documentation* [online], Axios Project, 2024, dostupno na: <https://axios-http.com/docs/intro> [Pristupljeno: 19.6.2025.]
- [8] C. Coyier, *A Complete Guide to Grid* [online], CSS-Tricks, 2023, dostupno na: <https://css-tricks.com/snippets/css/complete-guide-grid/> [Pristupljeno: 20.6.2025.]

SAŽETAK

Cilj ovoga rada bio je izraditi web aplikaciju čija je svrha elektroničko vođenje sjednica, a pri čemu su za razvoj frontend i backend dijela korištene moderne tehnologije. Za razliku od tradicionalnog načina održavanja sjednica, gdje je izraziti problem neučinkovitost i nepreglednost načina kojim se sjednice održavaju, ovakav način održavanja sjednice daje veći i pregledniji pristup korisnicima, ali i samoj organizaciji. Backend aplikacije razvijen je u Javi korištenjem Spring Boot okvira, uz primjenu JWT autentifikacije i WebSocket komunikacije. Za razvoj frontend dijela korišten je React s MobX bibliotekom koja upravlja stanjima unutar aplikacije. Korištenjem aplikacije omogućeno je stvaranje sjednica, dodavanje točki dnevnog reda za svaku sjednicu te mogućnost održavanja glasanja za prethodno definiranu točku. Također, prikazuju se i priloženi dokumenti za svaku točku te potom po završetku glasanja prikazuju se stupčasti grafikoni rezultata glasova. Kroz aplikaciju postignut je pouzdan i siguran način održavanja sjednica koji može poslužiti kao osnova za daljnji razvoj i korištenje u raznim područjima.

Ključne riječi: e-sjednica, glasanje, Java, React, web aplikacija.

ABSTRACT

Web application for conducting e-sessions

The aim of this thesis was to develop a web application intended for the electronic management of meetings, using modern technologies for both the frontend and backend parts. Unlike traditional ways of holding meetings, which are often inefficient and unclear, this approach offers users and the organization a more accessible and structured experience. The backend of the application was developed in Java using the Spring Boot framework, with the implementation of JWT authentication and WebSocket communication. The frontend was developed using React along with the MobX library for state management within the application. The application enables the creation of meetings, adding agenda items for each meeting, and conducting voting for each defined item. Additionally, documents attached to each item are displayed, and after the voting is completed, the results are presented using bar charts. The application provides a reliable and secure way of conducting meetings, offering a solid foundation for further development and use in various fields.

Keywords: e-session, Java, React, voting, web application

PRILOZI

P.1. Izvorni kod aplikacije

Dostupno na: <https://github.com/Luka-sasko/DiplomskiRad-Esjednica/tree/develop>